01 · Was ist Echtzeit

Einführung und Paradigmenwechsel · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 59

Willkommen zur Vorlesung Echtzeitsysteme!

- Neun Termine: je 4 Stunden Theorie und Praxis / Übungen
- Klausur am Ende des Semesters
- Fokus: moderne Multicore-Architekturen in der Industrie
- Hardware: STM32H755 (Dual-Core Cortex-M7/M4)

Notizen 1 / 59

Diese Folie ordnet den Termin ein, bevor Fachbegriffe eingeführt werden. EZS steht für Echtzeitsysteme, DHBW für die Duale Hochschule Baden-Württemberg. Die Zielgruppe ist der Bachelorstudiengang Elektro- und Informationstechnik, kurz EIT. Ein eingebettetes System, englisch Embedded System, steuert ein Gerät und hat oft keine gewohnte Bildschirmbedienung. Ein Mikrocontroller, MCU, enthält Prozessorkern, Speicher und Peripherie auf einem Baustein. Dual-Core bedeutet hier zwei Kerne im selben STM32H755: Cortex-M7 und Cortex-M4.
Die Reihe beantwortet vier Fragen. Was heißt rechtzeitig? Wie wird Arbeit im Programm strukturiert? Wie untersucht man Zeitgrenzen? Wie arbeiten die beiden Kerne zusammen? Der heutige Termin legt den Zeitvertrag und die ersten Programmstrukturen. Scheduling-Formeln und FreeRTOS folgen später.
Theorie und Labor gehören zusammen: Die Folien benennen Annahmen, das Board zeigt, was unter einer konkreten Konfiguration tatsächlich passiert. Eine Anwesenheitspflicht oder konkrete Klausurmodalitäten stehen auf dieser Folie nicht. Die sichtbare Aussage ist nur, dass am Semesterende eine Klausur im Modul Rechnersysteme folgt und neun Termine Theorie mit Praxis verbinden.
Für alle folgenden Beispiele brauchen wir dieselbe Hardware- und Annahmenbasis.

Lehrkonfiguration der Reihe

Gilt für VL1–VL9, sofern eine Folie nicht ausdrücklich abweicht. Kein Firmwareprojekt im Repository. Messungen bleiben ausstehend.

Größe	Wert	Art
Board / MCU	NUCLEO-H755ZI-Q, STM32H755ZIT6	A, UM2408 / DS12919
LED / Taster	LD2 = PE1, User-Button PC13	A, UM2408
Kerne, ab VL3	M7 Bare-Metal, M4 mit FreeRTOS über CMSIS-RTOS2	B
VL1-Labor	M4 zunächst ohne Betriebssystem, nur HAL-Zeitbasis	B
CPU-Takte	M7 400 MHz, M4 200 MHz, unter den Maxima 480/240	B, Maxima A
späterer RTOS-Tick	1 kHz; in VL1 noch nicht aktiv	B
UART	115200 Baud, 8N1	B
SRAM4	`0x38000000`, 64 KiB, non-cacheable für M7	A Adresse, B Attribut
HSEM	0 Boot, 1 Nutzdaten; ProcessID M7 `0x37`, M4 `0x34`	B
WWDG2	APB1 = 100 MHz, Prescaler 8	B, APB-Maximum je VOS bis 120 MHz ist A

A ist dokumentierte Hardware. B ist eine didaktische Annahme, keine Messung. C wäre ein Messwert; den gibt es in dieser Reihe nicht.

Notizen 2 / 59

Diese Tabelle ist die gemeinsame Annahmenbasis der Reihe, nicht schon ein gemessenes Projekt. Das Board NUCLEO-H755ZI-Q trägt den Mikrocontroller STM32H755ZIT6. Die Kerne Cortex-M7 und Cortex-M4 sitzen in diesem Baustein. A bezeichnet einen dokumentierten Hardwarefakt, B eine Lehrannahme, C einen tatsächlich erhobenen Messwert. C-Werte liegen nicht vor, weil im Repository kein Firmwareprojekt und keine Messung steckt.
LD2 ist die gelbe LED und hängt an PE1, also Port E, Pin 1. PC13 ist der Benutzer-Taster, nicht der Programmzähler. Ab Vorlesung 3 kann der M4 FreeRTOS über die Schnittstelle CMSIS-RTOS2 nutzen. CMSIS 6 heißt Common Microcontroller Software Interface Standard. CMSIS-RTOS2 ist nicht der FreeRTOS-Kernel selbst. In Vorlesung 1 bleibt die Anwendung auf dem M4 Bare-Metal, also ohne Betriebssystemkernel.

Notizen, Fortsetzung 2 / 59

Der M7 übernimmt in der gewählten Dual-Core-Konfiguration den System- und Boot-Anteil. Ein 1-kHz-RTOS-Tick ist heute noch nicht aktiv. Die HAL-Zeitbasis, auf der `HAL_Delay` wartet, existiert auch ohne Echtzeitbetriebssystem.
400 MHz und 200 MHz sind angenommene Kerntakte unter den spezifizierten Maxima 480 MHz und 240 MHz. Der Peripheriebustakt APB und der Timer-Kern-Takt können davon abweichen. UART ist die asynchrone serielle Schnittstelle; 8N1 bedeutet acht Datenbits, keine Parität, ein Stopbit. SRAM4 bei `0x38000000` ist ein konkreter 64-KiB-Speicherbereich, nicht „viermal RAM“. Die Adresse ist Hardwarefakt. Dass der M7 ihn nicht zwischenspeichert, muss die Konfiguration erst herstellen. Heute programmieren wir diesen Bereich noch nicht. HSEM ist eine Hardwaresemaphore zur Abstimmung der Kerne, keine Cache-Kohärenz. Die ProcessID ist ein frei vergebener 8-Bit-Besitzer, nicht die Hardware-Core-ID. WWDG2 ist eine Fenster-Watchdog-Instanz; ihre Funktion folgt später.
Mit dieser Basis betrachten wir den Ablauf des ersten Termins.

Folie 3 / 59

Agenda für heute

1. Organisatorisches und Spielregeln
2. Motivation und Definition von Echtzeit
3. Bare-Metal: Polling, Interrupts, Foreground/Background
4. Deep Dive: Cortex-M, NVIC und Tail-Chaining
5. Hardware STM32H755 und Toolchain
6. Laborübung am Board

Notizen 3 / 59

Die Agenda begründet die Reihenfolge, nicht eine minutengenaue Planung. Zuerst braucht der Zeitvertrag gemeinsame Begriffe, sonst lässt sich „zu spät“ nicht beurteilen. Danach folgen Softwaremuster, weil sie bestimmen, wann Arbeit überhaupt laufen darf. Der Interruptmechanismus erklärt anschließend, wie die Hardware eine laufende Schleife unterbricht. Board und Labor machen diese Kette an PE1 und einem Timer sichtbar.
Polling fragt zyklisch ab, ob etwas passiert ist. Ein Interrupt ist eine Hardwareanforderung, die den Kern unterbricht. Foreground/Background teilt kurze Interruptarbeit und längere Hintergrundarbeit. Cortex-M ist die Prozessorkernfamilie auf dem Board. Der NVIC, Nested Vectored Interrupt Controller, entscheidet, welche Anforderung angenommen wird. Tail-Chaining spart beim Übergang zwischen zwei Handlern das vollständige Zurück- und erneute Sichern. Diese Stichworte werden erst in ihren eigenen Blöcken erklärt. Jeder Block beantwortet eine Frage: Was ist eine Frist? Wie strukturiert man den Kontrollfluss? Was passiert im Kern beim Unterbrechen? Wie starten wir das Board? Belegte Zeitangaben für die Blöcke stehen hier nicht.
Bevor die fachliche Arbeit beginnt, werden Materialien und der organisatorische Rahmen eingeordnet.

Folie 4 / 59

Organisatorisches

- Prüfungsleistung: Klausur am Ende des Semesters (Modul Rechnersysteme)
- Materialien: Folien, Handouts, Übungen und Musterlösungen auf der Kurswebsite und in Moodle
- Labor-Hardware: Boards werden gestellt (STM32H755)
- Tools: empfohlene Lehrumgebung CubeIDE 2.x mit separatem CubeMX; CubeIDE 1.x integriert die Konfiguration

Notizen 4 / 59

Diese Folie sagt, wie Sie die Materialien benutzen. Die Präsentation hält die Kernaussagen. Das Handout enthält dieselben Folien plus diese Notizen und ist für die Nachbereitung gedacht. Übungen und Musterlösungen liegen auf der Kurswebsite und in Moodle. Die Klausur gehört zum Modul Rechnersysteme; weitere Modalitäten stehen hier nicht.
Das Board NUCLEO-H755ZI-Q wird gestellt. Die USB-Verbindung dient der Stromversorgung und dem Debuggen über den aufgelöteten ST-LINK. Die empfohlene Lehrumgebung ab STM32CubeIDE 2.0 trennt die Werkzeuge: STM32CubeMX konfiguriert Pins und Takte und erzeugt den Code, STM32CubeIDE importiert, übersetzt, lädt und debuggt. In CubeIDE 1.x steckte die Konfiguration noch in der IDE. Welche Variante im Laborraum installiert ist, prüfen Sie vor Ort. Der Kurs hat keine im Repository festgeschriebene, bereits erprobte Versionsnummer.
Praktische Arbeit macht das Zeitverhalten an einer konkreten Konfiguration sichtbar.
Für Theorie und technische Detailfragen werden unterschiedliche Quellen gebraucht.

Folie 5 / 59

Literatur und Quellen

- Hermann Kopetz: *Real-Time Systems* (Design Principles)
- Giorgio C. Buttazzo: *Hard Real-Time Computing Systems* (Scheduling)
- STMicroelectronics: Reference Manual STM32H745/755 (RM0399)
- FreeRTOS Documentation (online)
- Tipp: Datenblätter und Reference Manuals sind Ihre wichtigste Alltagsquelle

Notizen 5 / 59

Die Bücher und die ST-Dokumente beantworten verschiedene Fragen. Kopetz, *Real-Time Systems*, liefert Systemprinzipien: was ein Zeitvertrag ist und wie Fehlerfolgen gedacht werden. Buttazzo, *Hard Real-Time Computing Systems*, behandelt die zeitliche Planung mehrerer Aufgaben. FreeRTOS-Dokumentation wird erst relevant, wenn das Echtzeitbetriebssystem eingeführt ist; heute arbeiten wir noch ohne Kernel.
RM steht für Reference Manual, das Peripherie- und Systemhandbuch. RM0399 beschreibt den STM32H745/755, also Timer, Interruptweiterleitung und Speicher. PM steht für Programming Manual:

Notizen, Fortsetzung 5 / 59

PM0214 für den Cortex-M4, PM0253 für den Cortex-M7. UM ist das User Manual; UM2408 beschreibt das Nucleo-Board. AN ist eine Application Note, ein Anwendungshinweis für einen konkreten Ablauf. DS ist das Datasheet, das Datenblatt mit Bauteilgrenzen.
Eine kleine Suchstrategie: „Wo ist LD2 angeschlossen?“ steht im Boardhandbuch. „Wie setzt TIM2 seinen Interrupt?“ steht im Referenzhandbuch. „Was wird beim Ausnahme-Eintritt gesichert?“ steht im Programmierhandbuch des Kerns. Normwortlaute werden hier nicht zitiert.
Eine Alltagssituation zeigt als Nächstes, warum Zeit selbst Teil der Anforderung ist.

Folie 6 / 59

Einstieg: Diskussion

- Stellen Sie sich vor: Ihr ABS reagiert erst 200 ms später — ist das noch „richtig“ oder schon „falsch“? *(Diskussionsannahme:* 200 ms *ist keine reale ABS-Spezifikation.)*

Notizen 6 / 59

ABS steht für Antiblockiersystem. Es soll beim Bremsen verhindern, dass ein Rad blockiert, und dafür den Bremsdruck rechtzeitig verändern. Die Folie fragt, ob eine fachlich sinnvolle Reaktion trotzdem falsch sein kann, wenn sie erst 200 ms später kommt. Diese 200 ms sind eine frei gewählte Diskussionsannahme, keine Fahrzeugspezifikation und keine technische ABS-Grenze.
Eine brauchbare Musterantwort braucht drei Angaben: die Aufgabe, das Bezugsereignis und die zulässige Frist. Erst dann ist „zu spät“ definiert. Unter der angenommenen Frist von 200 ms wäre die verspätete Reaktion unzulässig, auch wenn der berechnete Druck fachlich richtig ist. Unter einer anderen, weiter gesetzten Frist könnte dieselbe Reaktion zulässig sein. Die Zahl selbst beweist nichts über ein reales Fahrzeug.
Frage: Kann eine fachlich richtige Reaktion trotzdem falsch im Sinne der Anforderung sein? Ja, sobald sie nach der spezifizierten Frist eintrifft. Der Wert des Ergebnisses und die Einhaltung der Frist sind zwei Prüfungen.
Solche zeitlichen Anforderungen treten auch außerhalb von Fahrzeugen auf.

Folie 7 / 59

Motivation: Warum Echtzeit?

- Steigende Komplexität in Embedded Systems
- Autonomes Fahren, Medizintechnik, Industrie 4.0 (Beispiele, keine pauschale Klassenzuordnung)
- Desktop-OS (Windows/Linux) liefern typischerweise keine harten Fristgarantien
- Timing-Fehler können sicherheitsrelevant sein — aber nicht jede Fristverletzung ist „katastrophal“
- Ziel: vorhersagbare Reaktionen innerhalb spezifizierter Fristen

Notizen 7 / 59

Ein Embedded System, ein eingebettetes System, reagiert auf technische Ereignisse, nicht nur auf die Geduld einer Benutzerin oder eines Benutzers. Eine Benutzerwartezeit ist eine Qualitätserwartung an der Oberfläche. Eine spezifizierte Reaktionsfrist ist ein Vertrag: Aufgabe, Ereignis und zulässige Dauer.
Die Beispiele zeigen Aufgaben, keine Branchenklassen. In der Industrie kann eine Steuerung einen Antrieb innerhalb eines Stellzyklus korrigieren. In der Medizintechnik kann ein Gerät einen Messwert rechtzeitig an eine Überwachung liefern. In der Mobilität kann ein Regler einen Stellwert vor dem nächsten Takt brauchen. Dieselbe Branche kann daneben eine unkritische Statusanzeige haben. Die Branche bestimmt die Echtzeitklasse nicht pauschal.
OS steht für Operating System, Betriebssystem. Ein typisches Desktop-System optimiert Durchsatz und Bedienbarkeit und gibt für beliebige Programme keine harte Fristgarantie. Ein entsprechend konfiguriertes System kann engere Grenzen nachweisen. Daraus folgt nicht die Aussage „Linux kann keine Echtzeit“.
Fehlerfolgen und der Fristvertrag hängen zusammen, sind aber verschiedene Fragen: Eine Frist kann hart sein, ohne dass jede Verletzung eine Katastrophe ist.
Diese Motivation führt zu einem verbreiteten Irrtum über Geschwindigkeit.

Folie 8 / 59

Der große Mythos der Echtzeit

Was viele denken:

- „Echtzeit = extrem schnell“
- „4 GHz ist echtzeitfähiger als 400 MHz“
- „Echtzeit = flüssiges Video“

Was es wirklich ist:

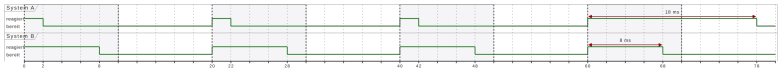
- Echtzeit = Rechtzeitigkeit (Fristvertrag)
- Ein 400-MHz-Controller mit Garantie kann echtzeitfähig sein
- Ein 4-GHz-PC ohne Garantie ist es oft nicht

Notizen 8 / 59

MHz und GHz nennen den Kerntakt, also wie viele Takte der Kern pro Sekunde ausführt. 1 MHz sind eine Million Takte je Sekunde, 1 GHz eine Milliarde. Dieser Takt bestimmt weder allein die End-to-End-Antwortzeit noch deren obere Grenze. Geschwindigkeit einer einzelnen Berechnung, Durchsatz vieler Berechnungen und eine garantierte rechtzeitige Reaktion sind drei verschiedene Eigenschaften.
Das Lehrbeispiel setzt eine Frist von 10 ms. Ein System, das im Mittel 1 ms braucht, aber einmal 50 ms braucht, verletzt diese Frist trotz des guten Mittelwerts. Ein 400-MHz-Controller ist nicht schon wegen seiner Bauart echtzeitfähig. Eine Garantie braucht benannte Annahmen: maximale Ausführungszeit, Wartezeit und zulässige Unterbrechungen.
Frage: Reicht ein Mittelwert von 1 ms für eine harte Frist von 10 ms? Nein. Der Mittelwert sagt nichts über den einzelnen Ausreißer. Ohne eine begründete obere Schranke bleibt die Frist unbewiesen.
Die nächste Folie macht denselben Unterschied an zwei konkreten Zeitfolgen sichtbar.

Folie 9 / 59

Mythos: Mittelwert ≠ harte Garantie



- Illustrative Einzelbeobachtungen, graue Fenster = Deadline $D = 10$ ms
- A: $2/2/2/18$ ms (Mittel 6 ms), B: $8/8/8/8$ ms (Mittel 8 ms)
- A ist im Mittel schneller als B — und verletzt trotzdem die Frist
- Vier Messungen bei B sind kein allgemeiner Nachweis; eine separat angenommene Schranke wäre z. B. $R \leq 9$ ms
- Rechtzeitigkeit braucht eine spezifizierte Obergrenze, nicht nur einen Mittelwert

Notizen 9 / 59

Die Abbildung zeigt zwei Folgen von Antwortzeiten und graue Deadlinefenster von je 10 ms. D ist die relative Frist. R ist die Antwortzeit vom Release bis zur Fertigstellung, soweit wir sie in den Notizen so nennen.
System A liefert 2, 2, 2 und 18 ms. Der Mittelwert ist (2+2+2+18)/4 = 6 ms. Der vierte Wert 18 ms liegt hinter dem Fenster und ist ein Deadline Miss, eine Fristverletzung. System B liefert viermal 8 ms. Der Mittelwert ist (8+8+8+8)/4 = 8 ms, also schlechter als bei A. In diesen vier Fällen hält B die Frist ein. Vier Erfolge beweisen keine allgemeine Obergrenze. Ein einzelner Miss widerlegt dagegen einen universellen Garantieanspruch, der für alle Ausführungen gelten soll.
Die zusätzliche Schranke R kleiner oder gleich 9 ms ist eine Lehrannahme mit 1 ms Reserve zur Deadline, keine Messung.
Ein besserer Mittelwert verhindert also keinen einzelnen Miss. Ob und warum ein solcher Miss unzulässig ist, legt die jeweilige Anforderung fest.

Folie 10 / 59

Konsequenzen von Timing-Fehlern

- Funktionale Korrektheit und Rechtzeitigkeit sind unabhängige Kriterien
- Diskussionsannahme Airbag (Zahlen illustrativ, nicht Spezifikation):
  - Crash erkannt (funktional ok)
  - Zündung ausgelöst (funktional ok)
  - Zündung 50 ms zu spät (zeitlich falsch unter angenommener Frist)
- Bei hartem Fristvertrag: Ergebnis nach der Deadline gilt als Anforderungsverletzung
- Nicht pauschal: bei weicher Echtzeit kann ein spätes Ergebnis noch Restnutzen haben

Notizen 10 / 59

Funktionale Korrektheit fragt, ob der Wert stimmt. Zeitliche Korrektheit fragt, ob er bis zur Frist vorliegt. Daraus entstehen vier Möglichkeiten: richtig und pünktlich, richtig und zu spät, falsch und pünktlich, falsch

Notizen, Fortsetzung 10 / 59

und zu spät. Nur die erste erfüllt beide Kriterien.
Die Airbaggeschichte ist qualitative Motivation. „50 ms zu spät“ ist eine illustrative Verspätung, keine reale Airbagspezifikation. Der genaue Bezugszeitpunkt der Frist steht in diesem Beispiel nicht. Unter einem harten Vertrag gilt die verspätete, sonst richtige Zündung als Anforderungsverletzung. Ein weiches Qualitätsziel kann ein spätes Ergebnis noch mit abnehmendem Nutzen bewerten. Die Klasse folgt aus dem Vertrag, nicht aus der Dramatik der Geschichte.
Daraus lassen sich überprüfbare Lernziele für den heutigen Termin ableiten.

Folie 11 / 59

Lernziele des heutigen Tages

1. Arbeitsdefinition von Echtzeit (Werte- und Zeitbereich) erklären
2. Harte, feste und weiche Echtzeit anhand Aufgabe+Frist unterscheiden
3. Metriken Latenz, Antwortzeit, Jitter und WCET sauber trennen
4. Polling, Interrupts und Foreground/Background zeitlich einordnen
5. Erkennen: Architektur hilft — Deadlines bleiben nachzuweisen

Notizen 11 / 59

Jedes Lernziel ist eine Handlung. Sie können einen Fristvertrag mit Aufgabe, Ereignis, Frist und Abschlusskriterium formulieren. Sie können eine Aufgabe als hart, fest oder weich einordnen, indem Sie fragen, was nach der Frist mit Anforderung und Nutzen geschieht. Sie können in einem Zeitdiagramm Release, Start, Fertigstellung, CPU-Bedarf und Antwortzeit zuordnen. Sie können eine Super-Loop zeitlich beurteilen, wenn die Maximalzeiten der Schritte benannt sind. Sie können den Weg eines Timer-Interrupts von der Anforderung bis zum Rücksprung erklären.
WCET, Worst-Case Execution Time, ist eine begründete obere Schranke der Ausführungszeit, nicht das bisher größte Messergebnis. RTOS, Real-Time Operating System, ist ein Echtzeitbetriebssystem und heute noch nicht aktiv. Bare-Metal bedeutet Programm ohne Betriebssystemkernel. Im Labor wird die LED-Reaktion sichtbar. Ein harter Fristnachweis braucht zusätzlich Annahmen und eine Schranke. Vollständige Scheduling-Theorie setzen wir dafür nicht voraus.
Die folgenden Folien beginnen mit einer gemeinsamen Arbeitsdefinition.

Folie 12 / 59

Was ist Echtzeit? (Definition)

- Zwei unabhängige Kriterien, beide müssen erfüllt sein:
  1. funktional korrekt — der Wert ist richtig (Wertebereich)
  2. rechtzeitig — der Wert liegt bis zur Frist vor (Zeitbereich)
- Keines der beiden Kriterien folgt aus dem anderen
- Historischer Bezug: „Echtzeitbetrieb“ in Anlehnung an DIN 44300 — Lehrzitat, kein geprüfter Normwortlaut
- Arbeitsdefinition (Lehrkontext): ein System erfüllt Echtzeitanforderungen, wenn spezifizierte Fristen unter den genannten Annahmen eingehalten werden
- Determinismus (Arbeitsbegriff): zeitliches Verhalten unter dokumentierten Bedingungen begrenzt vorhersagbar

Notizen 12 / 59

Ein Sensorwert kann im Wertebereich stimmen und im Zeitbereich zu spät sein. Ein Stellbefehl kann pünktlich ankommen und trotzdem den falschen Zahlenwert tragen. Keines der beiden Kriterien erzeugt das andere.
Ein einfacher Vertrag nennt die Aufgabe, das auslösende Ereignis, die Frist, das Abschlusskriterium und die Randbedingungen, etwa maximale Last oder erlaubte Unterbrechungen. Begrenzte Vorhersagbarkeit meint eine begründete Zeitschranke. Sie verlangt nicht, dass jeder Durchlauf exakt gleich lang dauert.
DIN steht für Deutsches Institut für Normung. Der Hinweis auf DIN 44300 ist ein historischer Bezug auf den Begriff Echtzeitbetrieb, kein hier geprüfter Normwortlaut. Verbindlich für den Kurs ist die Arbeitsdefinition auf der Folie.
Nach der Definition betrachten wir unterschiedliche Konsequenzen von Fristverletzungen.

Folie 13 / 59

Klassifizierung von Echtzeitsystemen

- Nicht jede Fristverletzung ist gleich zu bewerten
- Unterscheidung nach spezifizierter Konsequenz eines Deadline Miss:
  - Harte Echtzeit (Hard Real-Time)
  - Feste Echtzeit (Firm Real-Time)
  - Weiche Echtzeit (Soft Real-Time)
- Beispiele immer als konkrete Aufgabe + Frist, nicht als ganze Branche dauerhaft einer Klasse

Notizen 13 / 59

Die zentrale Frage lautet: Was passiert mit der Anforderung und mit dem Nutzen, nachdem die Frist

Notizen, Fortsetzung 13 / 59

verstrichen ist? Deadline Miss bedeutet, dass das Ergebnis nach dieser Frist fertig wird. Hart, englisch Hard Real-Time: der Vertrag erlaubt keinen Miss. Fest, Firm Real-Time: das einzelne verspätete Ergebnis ist wertlos, eine begrenzte Zahl von Misses kann aber spezifiziert sein. Weich, Soft Real-Time: ein spätes Ergebnis kann noch Restnutzen haben.
Ein Gerät kann alle drei Verträge enthalten. Eine Stromregelung kann hart sein, eine Sortierentscheidung fest und eine Statusanzeige weich. Die Branche sortiert diese Aufgaben nicht von allein. Der Entwurf muss die Miss-Toleranz und den Geltungsbereich ausdrücklich festlegen: für welche Aufgabe, welches Ereignis und welches Zählfenster.
Zuerst wird der Vertrag ohne zugelassene Fristverletzung betrachtet.

Folie 14 / 59

Harte Echtzeit (Hard Real-Time)

- Spezifikation darf die Frist nicht verletzen
- Deadline Miss = Anforderungsverletzung (unabhängig vom Mittelwert)
- Nutzenkurve (illustratives Modell): Nutzen nach Miss abrupt null oder negativ
- Beispielaufgabe: PWM-Update für Motorstromregelung vor nächstem PWM-Zyklus
- Frist: z. B. $D = 100\,\mu s$ (Lehrannahme)

> **Sicherheit ist möglicher Kontext, nicht Definition**
>
> Sicherheitsfolgen für Mensch oder Umwelt können einen harten Fristvertrag begründen — hart bleibt die Klasse aber auch dann, wenn ein Miss „nur“ die Anforderung verletzt.

Notizen 14 / 59

PWM, Pulse-Width Modulation, ist Pulsweitenmodulation: ein wiederkehrendes Pulsverhältnis stellt einen mittleren Stellwert ein. In der Lehraufgabe muss die Stromregelung den nächsten Stellwert vor dem nächsten PWM-Zyklus abgeschlossen haben. Die 100 µs sind eine Lehrannahme, keine Bauteilspezifikation. Die Frist läuft ab dem gewählten Bezugsereignis, etwa dem Beginn des Zyklus, und endet, wenn der neue Stellwert übernommen ist. Welches Ereignis genau gilt, muss der Vertrag nennen.
Die Nutzenkurve ist ein didaktisches Modell: Nach dem Miss fällt der Nutzen sprunghaft. Das verbindliche Kriterium bleibt die Anforderung, nicht die gezeichnete Kurve. Sicherheit kann begründen, warum ein Vertrag hart gewählt wird. Eine Katastrophe ist kein notwendiger Bestandteil der Definition.
Frage: Wird die Klasse weich, wenn ein einzelner Miss keinen Schaden auslöst? Nein. Der spezifizierte harte Vertrag ist trotzdem verletzt. Der Schaden betrifft die Folge, die Klasse den Vertrag.
Bei fester Echtzeit verliert das einzelne verspätete Ergebnis seinen Nutzen, während begrenzte Misses toleriert werden können.

Folie 15 / 59

Feste Echtzeit (Firm Real-Time)

- Ergebnis muss vor einem Übergabepunkt vorliegen
- Danach: Ergebnis für diese Aktion wertlos (Nutzen $= 0$), ohne zwingend Systemausfall
- Beispielaufgabe: Werkstück sortieren — Kamera klassifiziert, Weiche schleust aus
- Übergabepunkt 150 mm hinter der Kamera, Band 1 m/s $\rightarrow D = 150$ ms (Lehrannahme)
- Miss-Toleranz explizit festlegen, z. B. höchstens 1 verworfenes Teil je 1000 (Lehrannahme)
- Nutzenkurve (illustratives Modell): Sprung auf null nach der Deadline

Notizen 15 / 59

Die Weggröße nennen wir hier ell, damit sie nicht mit dem späteren Startzeitpunkt s verwechselt wird. ell = 150 mm = 0,15 m. Bei einer Bandgeschwindigkeit von 1 m/s ist die Laufzeit ell/v = 0,15 m / 1 m/s = 0,15 s = 150 ms. Das ist zunächst die Zeit, bis das Teil die Weiche erreicht. Kamera, Klassifikation und Weiche bilden die Kette. Der physische Übergabepunkt liegt 150 mm hinter der Kamera.
Der Vertrag muss den Endpunkt nennen. Ist der Endpunkt die abgeschlossene mechanische Umlenkung, gehören Bildaufnahme, Kommunikation, Aktuatorlaufzeit und Reserve in diese 150 ms. Die Entscheidung muss dann entsprechend früher fertig sein. Ist der Endpunkt nur die Entscheidung, braucht es eine frühere Entscheidungsfrist, bevor der Aktuator seine eigene Zeit verbraucht. Ein Ergebnis nach dem Vorbeilaufen wird für dieses Teil verworfen. Die Anlage muss deshalb nicht ausfallen.
„Höchstens 1 von 1000“ ist nur prüfbar, wenn das Fenster festliegt, zum Beispiel jedes zusammenhängende Fenster von 1000 Teilen. Daraus folgt keine statistische Zuverlässigkeitsgarantie. Alle Zahlen sind Lehrannahmen.
Bei weicher Echtzeit kann ein spätes Ergebnis noch Restnutzen besitzen.

Folie 16 / 59

Weiche Echtzeit (Soft Real-Time)

- Deadline-Miss mindert Qualität; Ausfall ist nicht zwingend
- Aktualität / Nutzen nimmt typischerweise ab (kein universelles Video-Nutzenprofil)
- Nutzenkurve (illustratives Modell): allmählicher Abfall — Form applikationsspezifisch
- Beispielaufgabe: Statusanzeige einer Temperatur mit Zielaktualität 1 s
- Fristverletzung: Anzeige veraltet — Qualitätsziel verfehlt, nicht automatisch Systemversagen

Notizen 16 / 59

Die Zielaktualität von 1 s ist eine Lehrannahme für eine Temperaturanzeige. Ein etwas älterer Wert kann noch informativ sein, weil sich die Temperatur meist nicht sprunghaft ändert. Der Nutzen fällt deshalb allmählich, und die Form dieses Abfalls ist anwendungsspezifisch, kein allgemeines Videoprofil.
Auch weiche Echtzeit braucht eine Qualitätsanforderung: eine Zielaktualität und einen festgelegten

Notizen, Fortsetzung 16 / 59

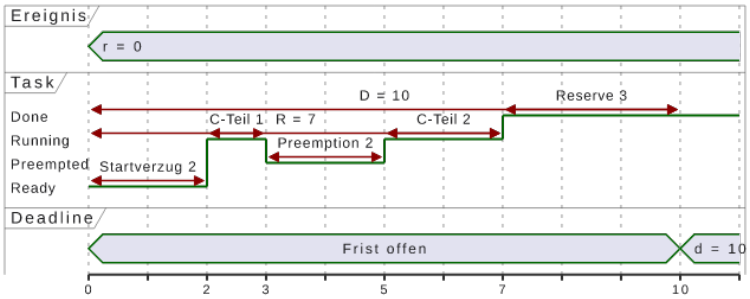
Umgang mit Überschreitungen. Neue Grenzwerte werden hier nicht als vorhandene Spezifikation ausgegeben. Typisches Verhalten, seltene Ausreißer und die vereinbarte Qualität gehören zusammen. Ein Entwurf, der nur den durchschnittlichen Fall betrachtet, muss trotzdem sagen, welche Ausreißer die Qualitätsanforderung noch erfüllt.
Die Klassen unterscheiden sich im Umgang mit verspäteten Ergebnissen. Um Verspätung überhaupt eindeutig zu erkennen, müssen wir Ereignis, Start, Fertigstellung und Frist auseinanderhalten.

Folie 17 / 59

Zeitstrahl-Vokabular

Lehrbeispiel (eine Job-Ausführung, Zeiten in ms):

- Ereignis / Release $r = 0$, Start $s = 2$, Fertig $f = 7$, absolute Deadline $d = 10$
- Ausführung 2–3 und 5–7; Präemption 3–5
- Startverzögerung $= 2$, CPU-Bedarf $C = 3$, Präemption $= 2$
- Antwortzeit $R = f - r = 7$, Fristreserve $= d - f = 3$
- Relative Deadline $D = d - r = 10$; sie ist nicht die Periode T
- Absolute/relative Deadline $\neq$ Periode T



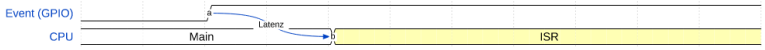
Notizen 17 / 59

Eine Task ist eine Aufgabe, die beispielsweise regelmäßig einen Sensorwert verarbeitet. Ein Job ist eine einzelne Ausführung dieser Aufgabe. In unserem Lehrbeispiel wird der Job bei r = 0 ms freigegeben. Release bedeutet, dass er ab diesem Zeitpunkt ausgeführt werden darf. Hier nehmen wir an, dass Release und auslösendes Ereignis zusammenfallen. Erst bei s = 2 ms bekommt der Job die CPU, die zentrale Verarbeitungseinheit. Die ersten 2 ms sind Wartezeit und keine eigene Rechenzeit.
Die Abbildung hat drei Spuren. „Ereignis“ markiert die Freigabe. „Task“ wechselt zwischen Ready, Running, Preempted und Done. „Deadline“ bleibt bis d = 10 ms offen. Von 2 bis 3 ms läuft der Job. Dann verdrängt andere Arbeit ihn; diese Verdrängung heißt Präemption. Von 3 bis 5 ms erhält er keine CPU-Zeit. Von 5 bis 7 ms läuft er wieder und ist bei f = 7 ms fertig. Der CPU-Bedarf C ist (3−2)+(7−5) = 3 ms. Die Antwortzeit R = f−r = 7 ms enthält Wartezeit und Präemption. Die absolute Deadline d = 10 ms ist ein Zeitpunkt. Die relative Deadline D = d−r = 10 ms ist die zulässige Dauer ab Freigabe. Die Reserve ist d−f = 3 ms. Fertigstellung genau auf der Deadline wäre bei diesem Vertrag noch rechtzeitig. Die Periode T ist der Abstand wiederholter Freigaben und nicht automatisch gleich D.
Als Nächstes betrachten wir, welche Verzögerung wir meinen, wenn wir von Latenz sprechen.

Folie 18 / 59

Latenz (Reaktionsverzögerung)

- Latenz: Verzögerung zwischen Ereignis und einer definierten Reaktion
- Interrupt-Latenz: Flanke am Pin bis erste Instruktion der ISR
- Kurze Hardware-Kette: Pin-Flanke $\rightarrow$ IRQ $\rightarrow$ erste Handlerinstruktion $\rightarrow$ HAL-Callback $\rightarrow$ GPIO
- Response Time R (Release $\rightarrow$ Completion): formal vertieft in VL2; $\neq$ Interrupt-Latenz
- Feste Latenz (konstant) $\neq$ Jitter (Schwankung)



Notizen 18 / 59

Latenz ist nur definiert, wenn Anfang und Ende genannt sind. GPIO, General-Purpose Input/Output, ist ein konfigurierbarer digitaler Ein- oder Ausgang. IRQ, Interrupt Request, ist die Interruptanforderung. ISR, Interrupt Service Routine, ist die Behandlungsroutine. HAL, Hardware Abstraction Layer, ist die Hardware-Abstraktionsschicht. Ein Callback ist eine Funktion, die HAL aus dem Handler aufruft.
Die Wellenform zeigt ein GPIO-Ereignis und danach den Wechsel der CPU von Main zur ISR. Der Pfeil „Latenz“ verbindet die Flanke mit dem ISR-Beginn. Das ist eine bewusst gewählte Gesamtlatenz der Anwendung nur dann, wenn der Endpunkt wirklich die erste Handlerinstruktion ist. Pin-Flanke bis sichtbare GPIO-Reaktion und die Antwortzeit eines ganzen Jobs haben andere Endpunkte.
Die idealisierten 12 Zyklen gelten für den Cortex-M4-Eintritt ab angenommener Interruptanforderung unter passenden Bedingungen, nicht für jeden Cortex-M-Kern und nicht für Flanke bis Callback. Pin-Synchronisierung, Peripherie, Weiterleitung, Maskierung, höhere Ausnahmen und die HAL-Kette können zusätzliche Zeit verursachen. Eine konstante Verzögerung ist etwas anderes als eine von Ausführung zu Ausführung wechselnde Verzögerung.
Die Schwankung zwischen mehreren Ausführungen wird als Jitter mit einer definierten Metrik beschrieben.

Folie 19 / 59

Jitter

Illustrative Messreihe (Sollperiode $T = 10$ ms):

	t_0	t_1	t_2	t_3	t_4
Ideal	0	10	20	30	40
Ist	0	$9{,}8$	20	$29{,}8$	40

- Perioden (Ist): $9{,}8 / 10{,}2 / 9{,}8 / 10{,}2$ ms
- Peak-to-Peak-Periodenjitter $= 10{,}2 - 9{,}8 = 0{,}4$ ms
- Phasenfehler zu Ideal: $0 / -0{,}2 / 0 / -0{,}2 / 0$ ms; Spanne $0{,}2$ ms (nicht mit Periodenjitter vertauschen)
- Latenzjitter separat: $\max(L) - \min(L)$
- Metrik nennen: Spanne / Peak-to-Peak in ms; Varianz in ms^2, Standardabweichung in ms
- Regler: Entwurfsfrage (Abtastmodell, Robustheit) — keine pauschale „Stabilitätszerstörung“



Notizen 19 / 59

Der Sollzeitpunkt ist der geplante Zeitpunkt, der Istzeitpunkt der beobachtete. Die Tabelle nennt die Istzeitpunkte 0; 9,8; 20; 29,8 und 40 ms bei einer Sollperiode von 10 ms. Die vier Istperioden sind 9,8−0 = 9,8 ms, 20−9,8 = 10,2 ms, 29,8−20 = 9,8 ms und 40−29,8 = 10,2 ms. Peak-to-Peak, die Spanne aus größtem minus kleinstem Wert, beträgt hier 10,2−9,8 = 0,4 ms. Das ist Periodenschwankung, nicht Phasenfehler.

Die Phasenfehler gegen die Idealzeitpunkte 0, 10, 20, 30 und 40 ms sind 0; −0,2; 0; −0,2 und 0 ms. Das negative Vorzeichen bedeutet einen früheren Zeitpunkt. Die Phasenspanne ist 0,2 ms. Die Wellenform zeichnet die idealen und die verschobenen Impulse getrennt. Latenzjitter verlangt wiederholte Messungen derselben Latenzdefinition. Varianz hat die Einheit ms², die Standardabweichung ms. Eine konstante Periode von 550 ms statt 500 ms ist im idealisierten Beispiel eine systematische Abweichung, keine Periodenschwankung.

Für harte Fristen genügt es nicht, Schwankungen nur zu beobachten. Wir brauchen eine belastbare obere Grenze.

Folie 20 / 59

Determinismus und WCET

- WCET: sichere Obergrenze der Ausführungszeit unter Worst-Case-Annahmen
- BCET: untere Grenze / minimale Ausführungszeit
- Deterministisch: unter dokumentierten Bedingungen begrenzt vorhersagbar (nicht: immer identische Laufzeit)
- Trennen: CPU-Bedarf C vs. präemptierte Zeit vs. Antwortzeit R
- Caches, Buskonflikte, Pfade und Interrupts erschweren enge Schranken

Notizen 20 / 59

WCET, Worst-Case Execution Time, ist eine sichere obere Schranke der Ausführungszeit unter benannten Annahmen. BCET, Best-Case Execution Time, ist die untere Grenze. Die tatsächliche Ausführungszeit eines Laufs liegt dazwischen. Ein Messmaximum ist nur der größte bisher gesehene Wert. Die Definition des CPU-Bedarfs muss sagen, ob Unterbrechungen herausgerechnet werden. Die Antwortzeit enthält zusätzlich Wartezeiten.
Ein kalter Cache muss Daten erst laden, ein warmer findet sie vor. Flash-Waitstates verlängern Zugriffe auf den nichtflüchtigen Programmspeicher. Unterschiedliche Programmpfade und Buskonflikte ändern die Dauer. Interrupts verlängern die verstrichene Zeit und können über veränderte Zustände auch den späteren CPU-Bedarf beeinflussen. Determinismus heißt hier begrenzte Vorhersagbarkeit unter dokumentierten Bedingungen, nicht identische Laufzeit in jedem Durchlauf.
Die nächste Folie zeigt, warum ein bislang größter Messwert noch keine solche Grenze beweist.

Folie 21 / 59

Messmaximum vs. WCET

Lehrwerte (erfunden, keine Boardmessung): beobachtetes Maximum 80 µs, sichere Obergrenze 120 µs.



- Messmaximum $\neq$ WCET: seltenere/ungünstigere Fälle können fehlen
- Annahmen notieren: Cold/Warm-Cache, Flash-Waitstates, Buslast, ausgeführte Pfade
- Vermeiden: „kein D-Cache = immer gleiche Laufzeit“
- Nicht: „WCET absichern nur durch extreme Tests“ — Analyse, Messung und Annahmen gehören zusammen (Vertiefung VL2)

Notizen 21 / 59

Das Flussdiagramm führt von beobachteten Zeiten über ein Messmaximum von 80 µs zu einer angenommenen sicheren Obergrenze von 120 µs. Beide Zahlen sind erfunden und keine Boardmessung. N ist die Anzahl beobachteter Läufe. Ungetestete Eingaben, andere Cachezustände und höhere Buslast können

Notizen, Fortsetzung 21 / 59

länger dauern als alles bisher Gesehene. Eine pauschale Reserve auf dem Messmaximum ist ebenfalls kein Beweis. Pfadwissen, Hardwareannahmen und eine passende Analyse oder Messmethodik können eine Schranke begründen. Solange diese Herleitung fehlt, gelten die 120 µs nur als gegebene Lehrschranke.
Nun werden die ähnlich klingenden Metriken noch einmal nebeneinander geordnet.

Folie 22 / 59

Zusammenfassung: Metriken

- Deadline d / D: Zeitpunkt bzw. relative Frist bis zum Ergebnis
- WCET: sichere Schranke für den CPU-Bedarf — Messmaximum allein reicht nicht
- Latenz: Verzögerung Ereignis $\rightarrow$ definierte Reaktion (fest $\neq$ schwankend)
- Antwortzeit R: Release bis Completion (inkl. Warte-/Präemptionszeiten)
- Jitter: Schwankungsmaß — Metrik nennen (z. B. Peak-to-Peak in ms)
- Goldene Regel: Vorhersagbarkeit unter Annahmen, nicht Durchsatz

Notizen 22 / 59

Am Zeitstrahlbeispiel gehören die Größen zu verschiedenen Messpunkten. Der CPU-Bedarf C = 3 ms zählt nur die beiden Laufabschnitte. Die Startverzögerung von 2 ms liegt zwischen Release und Start. Die Antwortzeit R = 7 ms läuft von der Freigabe bis zur Fertigstellung. Die relative Frist D = 10 ms ist die zulässige Dauer, die Reserve von 3 ms der Abstand bis zur absoluten Deadline. Eine einzelne Ausführung liefert noch keinen Jitter. WCET ist kein Synonym für die Antwortzeit: WCET begrenzt den CPU-Bedarf unter Annahmen, R enthält Wartezeit.
Frage: Wächst C zwangsläufig, wenn ein höherer IRQ unterbricht? Nicht zwangsläufig. Die verstrichene Antwortzeit wächst. C wächst nur, wenn die Unterbrechung in die gezählte Ausführungszeit hineingerechnet wird oder der Zustand danach mehr Arbeit erzeugt. Frage: Beweist ein niedriger Mittelwert die harte Frist? Nein. Ein einzelner Miss widerlegt die allgemeine Garantie, viele Erfolge ersetzen keine Schranke.
Damit kennen wir die Größen, die wir beurteilen wollen. Jetzt betrachten wir, wie unsere Programmstruktur die Warte- und Ausführungszeiten entstehen lässt.

Folie 23 / 59

Architekturen: Wie programmieren wir das?

- Wechsel von der Theorie zur Software-Architektur
- Ohne Betriebssystem: Bare-Metal
- Drei klassische Ansätze:
  1. Polling-Loops (Super-Loop)
  2. Interrupt-getriebene Systeme
  3. Foreground/Background-Systeme
- Keiner davon liefert automatisch einen Deadline-Nachweis

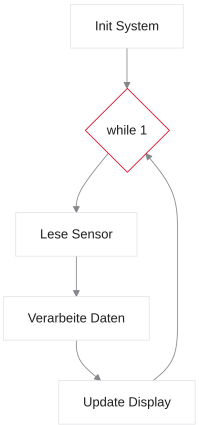
Notizen 23 / 59

Bare-Metal bedeutet ein Programm ohne Betriebssystemkernel. Bibliotheken und Interrupts sind trotzdem erlaubt. HAL kann auch in Bare-Metal verwendet werden. Polling fragt in einer Schleife ab. Eine rein interruptorientierte Bearbeitung reagiert in Handlern. Foreground/Background kombiniert die Hauptschleife mit Interrupt-Service-Routinen. Die drei Muster überlappen häufig. Ein RTOS ist ein späterer Strukturierungsansatz und heute noch nicht die Laborgrundlage.
Die Architektur beeinflusst, wann Arbeit laufen kann. Der Zeitvertrag muss unabhängig davon nachgewiesen werden. Kein Muster erzeugt allein eine Fristgarantie.
Das einfachste Kontrollflussmuster ist die nacheinander durchlaufene Super-Loop.

Folie 24 / 59

Architektur 1: Die Polling-Loop (Super-Loop)

- Simpelstes Muster: Endlosschleife `while(1)`
- Tasks strikt sequenziell (kooperativ)
- Status wird aktiv abgefragt („Polling“)
- Vorteil: simpel, wenig Overhead, leicht zu debuggen
- Qualifikation: zeitlich begrenzte kooperative Abläufe können Deadlines erfüllen



Notizen 24 / 59

`while(1)` ist eine Endlosschleife. Das Diagramm zeigt Initialisierung, dann die Runde Sensor, Verarbeitung und Display, zurück zum Schleifenkopf. Die Arbeitsschritte heißen hier Task nur als Arbeitsschritt, nicht als bereits existierender RTOS-Thread. Die Ausführung ist kooperativ: Eine Funktion gibt die Schleife erst nach ihrer Rückkehr frei. Overhead, der Verwaltungsaufwand, bleibt klein, weil kein Scheduler umschaltet.
Vorhersagbarkeit entsteht nur, wenn jede Teilaufgabe eine benannte Maximalzeit hat. Die maximale Rundenzeit bestimmt, wann eine Abfrage spätestens wieder drankommt. Aktives Warten prüft immer wieder, auch wenn nichts passiert ist, und kostet dafür Rechenzeit.
Am C-Beispiel wird sichtbar, an welcher Stelle UART, Taster und Display Zeit beanspruchen.

Folie 25 / 59

Code-Beispiel: Polling

```c
void main(void) {
    System_Init();
    while (1) {
        if (UART_Has_Data()) {    /* Polling */
            Process_Command();
        }
        if (Button_Pressed()) {   /* Polling */
            Toggle_LED();
        }
        Update_Display();
    }
}
```

- Super-Loop-Modell (Lehrannahme): Eingang $0{,}1$ ms, Komm. $0{,}2$ ms, Display 5 ms
- Maximale Runde $\approx 5{,}3$ ms; bei $D = 10$ ms ggf. ok
- Frage: Was passiert, wenn `Update_Display()` 50 ms braucht?

Notizen 25 / 59

`System_Init` richtet das System einmal ein. Danach läuft die Endlosschleife. `UART_Has_Data` prüft, ob auf der asynchronen seriellen Schnittstelle Daten liegen. Nur dann ruft `Process_Command` die Verarbeitung auf. `Button_Pressed` prüft den Taster und `Toggle_LED` schaltet die Leuchtdiode um. `Update_Display` aktualisiert die Anzeige immer. Diese Namen sind didaktische Hilfsfunktionen. Ihre vollständige Implementierung liegt nicht vor, und im Repository gibt es kein ausführbares Projekt. Der Code ist ein Muster.
Unter den Lehrannahmen maximale Eingangsprüfung 0,1 ms, Kommando 0,2 ms und Display 5 ms ist die modellierte Rundenzeit 0,1+0,2+5 = 5,3 ms. Das ist keine gemessene Firmwarezeit und kein Beweis, dass jede End-to-End-Antwort in 5,3 ms fertig ist. Kommt ein Ereignis unmittelbar nach seiner Abfrage, wartet es fast eine volle Runde. Für den Abschluss einer Antwort kann weitere Bearbeitung nötig sein.

Notizen, Fortsetzung 25 / 59

Ereignisphase, Position der Abfrage und Interruptlast gehören zur Aussage.
Wenn ein einzelner Schritt zu lange dauert, verschiebt er alle folgenden Abfragen.

Folie 26 / 59

Die Gefahr: Das Heavy-Load-Problem

- Eine lang laufende Funktion blockiert die gesamte Schleife
- Auf Eingänge kann nicht reagiert werden
- Fristbudget der Runde wird überschritten
- Typische Sünden: `delay(100)`, blockierende Warteschleifen
- Kurzer Puls zwischen zwei Kontrollen: Verlust hängt von Latch/FIFO/Signalbreite ab

```
void Update_Display(void) {
    /* ~50 ms Ausführungszeit */
    Draw_Pixels();
}
```

Worst-Case-Runde mit Display 50 ms: Budget $D = 10$ ms gerissen

Notizen 26 / 59

Während `Update_Display` etwa 50 ms rechnet, kommt die Schleife nicht zu UART und Taster zurück. Die CPU steht nicht. Sie rechnet nur an anderer Arbeit. Im einfachen Modell wird die Runde 0,1+0,2+50 = 50,3 ms. Bei einer relativen Frist D = 10 ms reicht dieses Budget nicht. Alle Zahlen bleiben Lehrannahmen.
`delay` und Busy-Wait-Schleifen blockieren denselben Kontrollfluss. Ein kurzer Eingangspuls kann zwischen zwei Abfragen verschwinden, wenn ihn niemand speichert. Ein Latch hält ein Ereignis fest, ein FIFO, First In First Out, puffert in Empfangsreihenfolge. Beides hat begrenzte Kapazität. Ereignisverlust und verspätete Bearbeitung sind verschieden: Ein gepuffertes Ereignis kann später noch korrekt bearbeitet werden, ein verlorenes nicht.
Polling bleibt sinnvoll, wenn Zeitbudgets und Ereignisverhalten dazu passen.

Folie 27 / 59

Fazit: Polling

- Vorteile: simpel, keine ISR-Races, feste Reihenfolge
- Nachteile: Priorisierung fehlt, lange Abschnitte gefährden Fristen, oft höherer Strom (aktives Warten)
- Einsatz: passende Zeitbudgets und wenige asynchrone Ereignisse
- Nicht: „Polling ist grundsätzlich ungeeignet für Echtzeit“

Notizen 27 / 59

Polling ist einfach und hat wenig Verwaltungsaufwand, solange die Teilaufgaben kurz und ihre Maximalzeiten bekannt sind. „Keine ISR-Races“ gilt für das einfache Modell ohne asynchronen Zugriff einer Interruptroutine auf gemeinsame Daten. DMA, Direct Memory Access, oder der zweite Kern können trotzdem gleichzeitig auf Speicher zugreifen, falls sie beteiligt sind. Eine feste Reihenfolge erzeugt nicht automatisch eine feste Laufzeit. Kurze kooperative Schritte können wichtigere Arbeit nach vorn ziehen, auch ohne präemptive Taskpriorisierung. Aktives Abfragen kostet Energie, weil der Kern nicht wartet, bis etwas passiert.
Wenn die nächste Abfrage zu spät kommt, kann ein Hardwareereignis eine frühere Reaktion auslösen. Dafür muss die CPU ihren aktuellen Ablauf unterbrechen.

Folie 28 / 59

Architektur 2: Interrupts zur Rettung

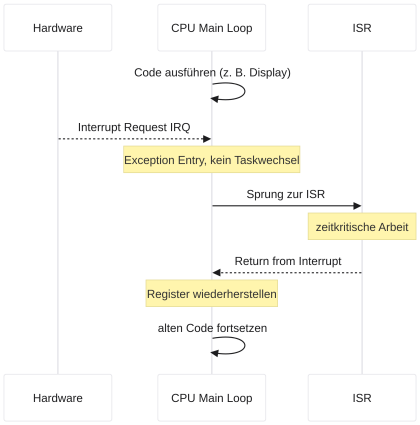
- Interrupt: Hardware-Signal, das die CPU unterbricht
- Kontext sichern, Sprung in die ISR
- Nicht mehr fragen — die Hardware meldet sich
- Ereignisse priorisierbar (z. B. Not-Halt vor Display)
- Interrupts erzeugen nicht automatisch Determinismus oder Fristnachweise

Notizen 28 / 59

Eine Interruptanforderung meldet ein Hardwareereignis. Der Kern prüft, ob sie zulässig ist, sichert Register und Rücksprunginformation, führt den Handler aus und setzt danach die unterbrochene Arbeit fort. IRQ ist die Anforderung, ISR die Behandlungsroutine. Auf demselben Kern laufen Main-Loop und ISR nicht parallel. Maskierung oder eine dringendere Ausnahme können die Reaktion verzögern. Ein Not-Halt ist hier nur ein Priorisierungsbeispiel. Ein einzelner IRQ ist keine funktionale Sicherheitsgarantie.
Die Hardware meldet das Ereignis. Ob der Kern es sofort annimmt, hängt von Maskierung, Priorität und bereits laufenden Handlern ab. Deshalb ist „die Hardware rettet die Frist“ nur dann richtig, wenn Zulassung und Handlerdauer im Budget liegen.
Das Sequenzdiagramm zeigt die Unterbrechung im zeitlichen Ablauf.

Folie 29 / 59

Der Ablauf eines Interrupts



Notizen 29 / 59

Das Sequenzdiagramm hat drei Spuren: Hardware, CPU beziehungsweise Main und ISR. Die ISR-Spur ist eine Darstellungsrolle, kein zweiter parallel rechnender Prozessor. Zuerst führt die CPU Hintergrundcode aus, zum Beispiel die Displayfunktion. Die Hardware sendet einen IRQ. Der Kern akzeptiert ihn und sichert den nötigen Zustand. Die Notiz im Diagramm heißt Exception Entry und meint diesen Ausnahme-Eintritt, nicht die Auswahl eines anderen RTOS-Tasks. Danach springt die CPU in die ISR, erledigt die zeitkritische Arbeit und kehrt zurück. Die Register werden wiederhergestellt, und der alte Code läuft weiter. Rückkehr heißt Fortsetzung der unterbrochenen Arbeit. Sie wird hier nicht als „genau derselbe einzelne Maschinenbefehl“ pauschalisiert, weil Pipeline und unteilbare Abläufe Details haben, die das Programmierhandbuch beschreibt.
Ein STM32-Timer liefert ein konkretes Beispiel für diesen Ablauf.

Folie 30 / 59

Code-Beispiel: Timer-Interrupt (STM32 HAL)

Setup in `main()`:

```
HAL_TIM_Base_Start_IT(&htim2);

while (1) {
    Update_Display(); /* Hintergrund */
}
```

Callback (ISR):

```
void HAL_TIM_PeriodElapsedCallback(
    TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        Toggle_LED(); /* zeitkritisch */
    }
}
```

Notizen 30 / 59

HAL ist die Hardware-Abstraktionsschicht. TIM2 ist die Timerinstanz 2. `htim2` ist das Software-Handle dieser Instanz, `TIM_HandleTypeDef` sein Typ. Der Adressoperator `&` übergibt die Adresse des Handles. `HAL_TIM_Base_Start_IT` startet den Timer im Interruptbetrieb. Das Suffix `_IT` kennzeichnet diesen Modus. Der Aufruf setzt voraus, dass Takt, Timer und NVIC bereits initialisiert sind. Die Kette lautet: TIM2-Update, dann `TIM2_IRQHandler`, dann `HAL_TIM_IRQHandler`, dann `HAL_TIM_PeriodElapsedCallback`. Der Callback läuft im Handlerkontext und ist kein eigener Task. Der Zeiger `htim` und der Zugriff `htim->Instance` prüfen, ob das Ereignis von TIM2 kommt, weil dieselbe Funktion mehrere Timer bedienen kann. `Toggle_LED` ist eine Hilfsfunktion. Danach läuft der Hintergrund weiter.
Der Callback kann die LED während langer Hintergrundarbeit bedienen. Weil er an unerwarteten Stellen in den Kontrollfluss eingreift, müssen wir gemeinsame Daten sorgfältig behandeln.

Folie 31 / 59

Gefahren von Interrupts

- Race Conditions bei gemeinsamen globalen Variablen
- Lange ISRs blockieren niederpriore IRQs und die Main-Loop
- Budget je ISR festlegen: kurz halten, kein `printf`, keine Delays, Maskierungsdauer begrenzen
- Jitter: laufende höhere IRQs verzögern niederpriore ISRs
- Flag-Race (Lehrfall): Main liest Flag → neues Event → Main löscht Flag — `volatile` allein reicht nicht

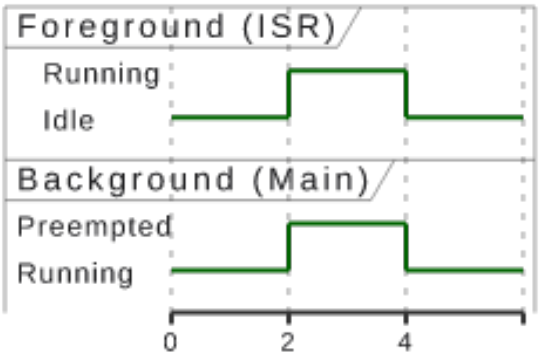
Notizen 31 / 59

Eine Race Condition ist ein Wettlauf um gemeinsame Daten, dessen Ergebnis von der zeitlichen Verschränkung abhängt. `volatile` sagt dem Compiler, dass sich ein Wert außerhalb des sichtbaren Ablaufs ändern kann. Es macht eine Folge weder atomar noch zu einem Zähler. Atomar heißt: der Schritt wird nicht in der Mitte unterbrochen. Ein kritischer Abschnitt ist der Programmbereich, der dafür geschützt werden muss.
Der Flag-Race läuft so: Ein Ereignis setzt das Flag auf 1. Die Hauptschleife liest 1. Bevor sie löscht, setzt ein weiteres Ereignis wieder 1. Die Hauptschleife schreibt 0 und löscht damit auch die Information über das zweite Ereignis. Ein kurzer geschützter Austausch kann diese Lücke schließen, zählt aber weiterhin nicht automatisch mehrere Ereignisse. Dafür braucht es einen Zähler oder Puffer. Die Dauer der Maskierung, also der Sperre weiterer Interrupts, gehört ins Zeitbudget.
Blockierende `printf`- oder Delay-Aufrufe im Handler halten den Kern fest und verzögern andere Arbeit. Spezielle nichtblockierende Ausgaben sind davon zu unterscheiden.
Eine bewährte Aufgabenteilung hält den Handler kurz und verlegt längere Arbeit in den Hintergrund.

Folie 32 / 59

Architektur 3: Foreground / Background

- Kombination: Main-Loop + Interrupts
- Background: unkritische Arbeit (`main`)
- Foreground: zeitkritische ISRs
- ISR setzt Auftrag; lange Main → Antwortzeit kann groß werden
- Standard für kleine bis mittlere Bare-Metal-Projekte



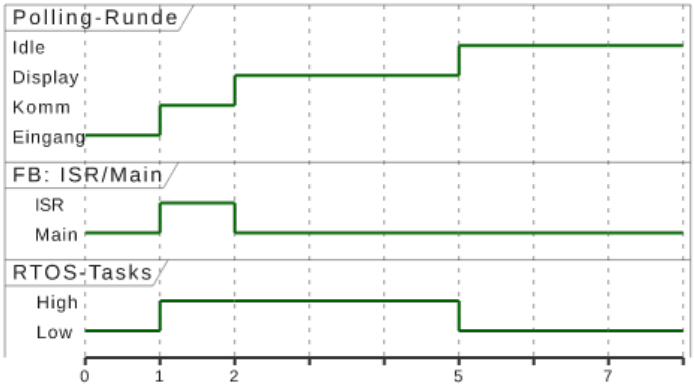
Notizen 32 / 59

Foreground bezeichnet hier die Interruptroutinen, Background die Hauptschleife. Ein Ereignis wird in der ISR kurz erfasst und als Auftrag übergeben. Die Hauptschleife verarbeitet ihn später und erzeugt das Ergebnis. Eine schnelle ISR garantiert nur ihre eigene frühe Reaktion. Die vollständige Antwort kann auf lange Hintergrundarbeit warten. Bei mehreren Aufträgen genügt ein einzelnes Boolean nicht, weil ein zweites Ereignis das erste überschreiben kann. Das Timer-LED-Labor ist der Sonderfall, in dem die kleine Reaktion vollständig im Handler erledigt werden kann.
Der Zeitachsenvergleich macht diese Unterschiede in Start und Abschluss sichtbar.

Folie 33 / 59

Drei Architektur-Zeitachsen

Lehrskizze derselben Frist $D = 10$ ms:



- Polling: Reaktion erst in der nächsten Abfrage der Runde
- Foreground/Background: ISR reagiert sofort, Arbeit oft erst in Main
- RTOS: Scheduler wählt Ready-Tasks — Fristen trotzdem nachweisen

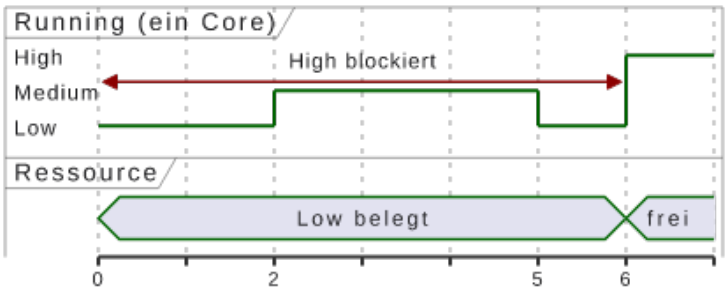
Notizen 33 / 59

Die drei Spuren beginnen am selben Referenzzeitpunkt und haben dieselbe relative Frist D = 10 ms. Ein fairer Vergleich setzt dieselbe Aufgabe und denselben Abschluss voraus. Beim Polling wartet die Arbeit bis zur nächsten Abfrage. Foreground/Background erfasst früh und verarbeitet möglicherweise später in der Hauptschleife. Ein RTOS kann einen ausführungsbereiten Task nach Prioritätsregeln wählen. Ready heißt ausführungsbereit, Running heißt, dass dieser Kontext die CPU hat. Der Scheduler ist die Komponente, die diese Auswahl trifft. „Sofort“ meint einen kurzen Handlerstart, nicht eine zeitlose Reaktion. Die gezeichneten Längen sind Lehrskizzen, keine Boardmesswerte.
Mit mehr Aufträgen wächst der Aufwand, diesen Kontrollfluss selbst zu verwalten.

Folie 34 / 59

Grenzen der klassischen Architekturen

- Warum reicht Foreground/Background oft nicht mehr?
- Spaghetti-Falle: ISRs setzen Flags, Main pollt endlos
- Bei vielen Tasks verliert man den Überblick über den Kontrollfluss
- Priorisierung und Schlafen von Tasks werden nötig
- Teaser Prioritätsinversion (Task-Kontext): Low hält Ressource, High blockiert, Medium verzögert Low — nur eine Running-Spur; kein Mutex in ISR → VL4



Notizen 34 / 59

Sobald zwei oder drei Aufgaben Flags, Zustände und eine gewünschte Reihenfolge teilen, wird eine große Hauptschleife unübersichtlich. „Flag-Suppe“ meint dann unklare Auftragszahl, unklare Bearbeitungsreihenfolge und schlecht sichtbare Wartezeiten. Bare-Metal bleibt analysierbar, wenn der Entwurf begrenzt bleibt. Eine allgemeine Unmöglichkeit folgt daraus nicht. Der Hinweis auf Prioritätsinversion gehört zum späteren Task- und Ressourcenkontext und ist hier nur Vorschau. Die Stufen Low, Medium und High sind Taskprioritäten und nicht die Unterprioritäten des NVIC.
Ein RTOS übernimmt einen Teil dieser expliziten Verwaltung durch Tasks und einen Scheduler.

Folie 35 / 59

Der Paradigmenwechsel

- Wenn Bare-Metal nicht skaliert: RTOS (Real-Time Operating System)
- RTOS: Strukturierungswerkzeug (Tasks, Prioritäten, Blockieren) — kein automatischer Deadline-Nachweis
- Bare-Metal kann ereignisgetrieben schlafen (WFE/Idle) — nicht nur aktives Polling
- Monolith → unabhängige Tasks unter Scheduler-Kontrolle

Notizen 35 / 59

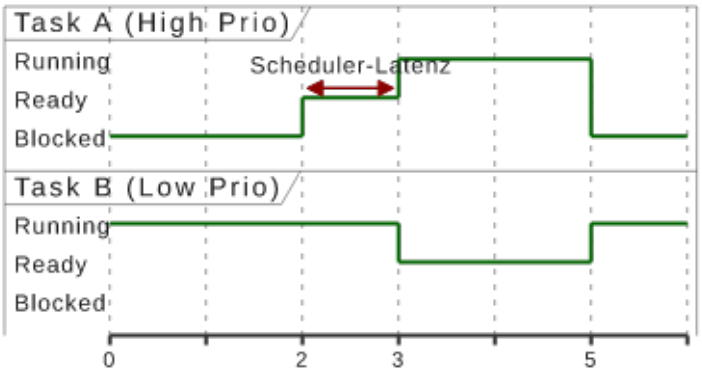
RTOS steht für Real-Time Operating System, Echtzeitbetriebssystem. Eine Task ist eine wiederkehrende oder dauerhafte Arbeitseinheit mit eigener Priorität. Blockieren heißt, auf ein Ereignis zu warten, ohne

Notizen, Fortsetzung 35 / 59

selbst die CPU auszuführen. Schlafen ist ein zeitlich begrenztes Warten. Der Scheduler wählt aus den bereiten Tasks aus. Präemptives Umschalten entzieht einem laufenden Task die CPU. Das unterscheidet sich vom freiwilligen Rückkehren eines Super-Loop-Schritts.
`HAL_Delay` in der Bare-Metal-Schleife blockiert den Hauptschleifen-Kontrollfluss und wartet auf die HAL-Zeitbasis. Ein blockierter RTOS-Task beansprucht dagegen keine eigene laufende Ausführung. Auch Bare-Metal kann ereignisgetrieben warten. WFE, Wait For Event, wartet auf ein Ereignis und ist kein austauschbares Boot-Protokoll. Verlässliche Fristen brauchen auch mit RTOS begrenzte Ausführungszeiten und passende Prioritäten.
Die nächste Zeitachse zeigt die Zustände bei einem geweckten Task.

Folie 36 / 59

Vorschau: Das Task-Modell



- Task B rechnet; Event weckt Task A
- RTOS entzieht Task B die CPU (Präemption) und gibt sie Task A
- Pro Kern rechnet höchstens ein Task-Kontext. Ein Handler belegt die CPU ebenfalls. Ready heißt ausführungsbereit, nicht bereits rechnend
- Ob Deadlines gehalten werden, bleibt Analyse-/Messaufgabe (VL2)

Notizen 36 / 59

Die Abbildung zeigt einen Kern. Task B rechnet. Ein Ereignis macht Task A bei t = 2 bereit. Ready bedeutet ausführungsbereit, nicht bereits rechnend. Running bei t = 3 zeigt, dass A erst danach die CPU erhält. Diese eine Millisekunde ist eine Lehrskizze der Verzögerung zwischen Bereitwerden und Laufbeginn, kein gemessener Scheduler-Overhead. Die Verzögerung kann Anteile des Schedulers, der Sicherung und höherer Handler enthalten. Unter der angenommenen Prioritätsregel verdrängt A den laufenden Task B. Zwei Task-Kontexte rechnen auf diesem Kern nicht gleichzeitig. Während eine ISR läuft, bedient die CPU den Handler, auch wenn der Kernel einen Task als aktuellen Task führt.
Taskzustände und Prioritäten beschreiben die Softwareentscheidung. Den Zustand beim Unterbrechen und Fortsetzen trägt jedoch der Prozessorkern; deshalb sehen wir uns dessen Mechanismus an.

Folie 37 / 59

Deep Dive: Die ARM Cortex-M Architektur

- Um Interrupts zu beherrschen, müssen wir die CPU verstehen
- Der STM32H755 enthält zwei Arm-Kerne, Cortex-M7 und Cortex-M4. AMP heißt Asymmetric Multiprocessing: unterschiedliche Programme auf den Kernen
- Cortex-M ist ein Produktfamilienname, keine geprüfte Akronym-Auflösung
- 32-Bit-**Programmier- und Registermodell**; nicht jede Leitung auf dem Chip ist 32 Bit breit
- MPU (Memory Protection Unit): Speicherschutz ohne Adressübersetzung. Eine MMU übersetzt zusätzlich Adressen. Beides beweist keine Frist
- Ein überschaubarer Interruptmechanismus erleichtert die Analyse. Identische Laufzeiten folgen daraus nicht
- PM0214 gilt für den Cortex-M4, PM0253 für den Cortex-M7

Notizen 37 / 59

Drei Ebenen sind zu trennen. Arm liefert den Kern. ST baut daraus den Mikrocontroller STM32H755 mit Speicher und Peripherie. Das Nucleo-Board NUCLEO-H755ZI-Q trägt diesen Baustein und die Anschlüsse. M7 und M4 sind zwei Kerne im selben Chip. AMP, Asymmetric Multiprocessing, beschreibt die gewählte asymmetrische Aufteilung. Cortex-M ist ein Produktfamilienname. Das M wird hier nicht als offizielle Akronym-Auflösung ausgegeben.
32 Bit betrifft das grundlegende Register- und Programmiermodell. ALU, Arithmetic Logic Unit, ist die arithmetisch-logische Einheit in diesem Modell. Nicht jede Verbindung auf dem Chip ist 32 Bit breit. Die MPU, Memory Protection Unit, schützt Speicherbereiche und übersetzt keine Adressen. Eine MMU, Memory Management Unit, übersetzt zusätzlich virtuelle in physische Adressen. Weder eine MPU noch der Verzicht auf eine MMU erzeugt einen Fristnachweis. Ein überschaubarer Interruptmechanismus kann die Analyse unterstützen. Identische Laufzeiten folgen daraus nicht. PM0214 ist das Programmierhandbuch des Cortex-M4, PM0253 das des Cortex-M7.
Zunächst betrachten wir die Register, deren Zustand beim Unterbrechen erhalten bleiben muss.

Folie 38 / 59

Das Register-Set der CPU

Register	Rolle
R0–R12	Allgemeine Datenregister
R13 (SP)	Stack Pointer
R14 (LR)	Link Register
R15 (PC)	Program Counter
xPSR	Statusregister (Flags)

- 16 Basis-Register à 32 Bit
- Bei Exception Entry: **nicht** alle 16 automatisch auf den Stack

Notizen 38 / 59

Register sind der kleine, schnelle Zustandsspeicher des Kerns. R0 bis R12 halten Daten und Adressen. R13 ist der Stack Pointer SP, der Stapelzeiger auf den Speicherbereich zum Sichern von Werten. R14 ist das Link Register LR mit Rücksprunginformation. R15 ist der Program Counter PC, der auf die Programmausführung zeigt. Dieses PC ist nicht Port-C-Pin 13 und nicht der Personal Computer. xPSR fasst Statusflags, Ausführungszustand und Ausnahmeinformation zusammen und kommt zusätzlich zum Basisregistersatz R0 bis R15 hinzu.
Die ISR benutzt denselben Registersatz. Nicht gesicherte Änderungen könnten die Hauptschleife beschädigen. Die Hardware sichert beim Eintritt aber nicht automatisch alle Register. Die nächste Tabelle zeigt den genau definierten einfachen Hardware-Stackframe.

Folie 39 / 59

Exception Entry (nicht „voller Taskwechsel“)

- Interrupt unterbricht den laufenden Thread
- Hardwaregesteuerter **einfacher** Stackframe (ohne FPU):

Wortindex	Byteoffset	Inhalt
0…7	0, 4, …, 28	R0, R1, R2, R3, R12, LR, PC, xPSR

- Grundgröße: $8 \times 4 = 32$ Byte (plus ggf. Alignment)
- R4–R11: bei Bedarf durch Compiler/Software (nicht automatisch)
- LR im Handler enthält typisch **EXC_RETURN** — kein gewöhnlicher Funktionsrücksprung

Notizen 39 / 59

Eine Exception ist der Oberbegriff für die Ausnahmebehandlung des Kerns. Ein IRQ ist eine mögliche Quelle. Im heutigen Bare-Metal-Programm läuft die Hauptschleife im Thread Mode. Der Handler läuft im Handler Mode. Es wird kein anderer RTOS-Task ausgewählt. Nach dem Handler wird die unterbrochene Arbeit fortgesetzt.
Der einfache Stackframe enthält acht 32-Bit-Wörter: R0, R1, R2, R3, R12, LR, PC und xPSR. Die Byteoffsets laufen von 0 bis 28 in Schritten von 4. Acht mal 4 Byte sind 32 Byte. Der gespeicherte PC merkt sich die Fortsetzungsstelle, das gespeicherte xPSR den Status. Das gestackte LR gehört zur unterbrochenen Arbeit. Das LR im Handler enthält typischerweise EXC_RETURN, eine kodierte Kennung für die Art des Rücksprungs, keinen normalen Codeadresszeiger. R4 bis R11 sichert bei Bedarf der Compiler, nicht dieser Hardwareframe. Alignment und ein Gleitkomma-Kontext der FPU, Floating-Point Unit, sind Erweiterungen und gehören nicht zum einfachen Lehrmodell.
Diesen Hardwareanteil ordnen wir nun in den vollständigen Weg bis zur HAL-Anwendung ein.

Folie 40 / 59

Von der Pin-Flanke zum HAL-Callback



- Hardware-Entry und HAL-Software farblich/logisch trennen
- Idealisiert oft ≈ 12 Entry-Zyklen (Cortex-M4, passende Annahmen) — **keine** Flanke-bis-Callback-Garantie

Notizen 40 / 59

Das Flussdiagramm trennt den rot umrandeten Hardware-Eintritt von den grau hinterlegten Softwareschritten. Die Kette beginnt mit der Pin-Flanke und dem Peripherieflag, geht über NVIC-Pending und Exception Entry zur ersten Handlerinstruktion, dann durch den vom Compiler eingefügten C-Prolog und den HAL-Handler zum Callback und gegebenenfalls zu einer GPIO-Änderung. Eine Pinmessung am Anfang und ein GPIO-Marker im Callback erfassen mehr als den reinen Hardware-Eintritt.
Im idealisierten Cortex-M4-Modell sind 12 Eintrittszyklen bei angenommenen 200 MHz rechnerisch 12 / 200e6 s = 60 ns. Das ist eine Umrechnung mit Lehrannahmen, kein Boardmesswert und keine Flanke-bis-Callback-Garantie.
Der NVIC entscheidet, ob und in welcher Reihenfolge Anforderungen angenommen werden.

Folie 41 / 59

Der NVIC: Herr der Interrupts

- NVIC = Nested Vectored Interrupt Controller
- Sitzt im Cortex-M-Kern, eng mit der CPU verzahnt
- **Architekturgrenze** Cortex-M: bis zu 240 externe IRQs
- **STM32H755-Implementierung:** bis zu ca. 150 maskierbare Kanäle, 16 Prioritätsstufen — **pro Kern eigener NVIC**
- Vectored: direkter Sprung über Vector Table

Notizen 41 / 59

NVIC steht für Nested Vectored Interrupt Controller. Nested heißt, dass eine dringendere zulässige Anforderung eine weniger dringende unterbrechen darf. Vectored heißt, dass die Vector Table einer Ausnahme direkt eine Handleradresse zuordnet. Sie ersetzt die spätere HAL-Statusprüfung nicht. Interrupt Controller ist die Einheit, die Anforderungen annimmt. Die Architektur erlaubt bis zu 240 externe IRQs. Die konkrete STM32H755-Implementierung nennt in den Kursfolien etwa 150 maskierbare Kanäle je Kern, einschließlich der nötigen Vorsicht bei Reservierungen und kernspezifischen Unterschieden. Die Zahl wird nicht künstlich als exakt ausgegeben. Jeder Kern hat einen eigenen NVIC. Vier implementierte Prioritätsbits ergeben 16 Stufen.
Die numerische Prioritätsrichtung ist für die richtige Konfiguration entscheidend.

Folie 42 / 59

Prioritäten im NVIC

- Nested: höhere Dringlichkeit darf niedere ISR unterbrechen
- Falle: **kleinere Zahl = höhere Priorität**
- Priorität 0: höchste *konfigurierbare* Priorität
- Feste Systemausnahmen (z. B. NMI) gesondert einordnen
- Zustände getrennt denken: Enabled / Pending / Active; Peripherieflag $\neq$ NVIC-Pending

Notizen 42 / 59

Bei den konfigurierbaren Interruptprioritäten gilt: kleinere Zahl, höhere Dringlichkeit. Priorität 0 ist die höchste konfigurierbare Stufe, aber nicht höher als alle fest priorisierten Systemausnahmen. NMI, Non-Maskable Interrupt, ist eine nicht mit den üblichen Masken sperrbare Systemausnahme. HardFault ist der Name einer Fehlerausnahme. Enabled heißt freigeschaltet, Pending anstehend, Active derzeit in Bearbeitung. Ein Interrupt kann je nach Lage erneut pending werden. Das Peripherieflag und das NVIC-Pending sind getrennte Zustände.
Im Lehrbeispiel hat TIM2 die Präemptionspriorität 5 und UART die Priorität 8. TIM2 darf den UART-Handler unterbrechen, umgekehrt nicht, sofern keine Maskierung und keine höhere Ausnahme dazwischenliegen. Priorität allein erzeugt keine obere Latenzgrenze.

Notizen, Fortsetzung 42 / 59

Beim Priority Grouping wird die Prioritätszahl in zwei unterschiedliche Rollen aufgeteilt.

Folie 43 / 59

Preemption Priority vs. Subpriority

- Prioritäts-Bits lassen sich aufteilen (Grouping)
- Preemption Priority: darf ein IRQ einen laufenden IRQ unterbrechen?
- Subpriority: bei gleichzeitigem Warten — wer zuerst?
- Lehrbeispiel, Gruppierung 2+2, nicht automatisch die HAL-Voreinstellung: A $= (1, 0)$, B $= (1, 3)$, gleiche Präemptionspriorität

Lage	Was gilt
A und B beide pending	A gewinnt die Auswahl, weil die Unterpriorität 0 vor 3 liegt
B bereits active	A unterbricht B nicht; die Unterpriorität ist kein Präemptionsrecht

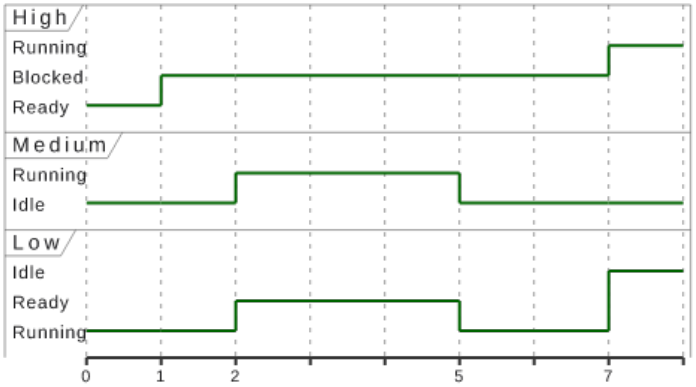
Notizen 43 / 59

Die Präemptionspriorität entscheidet, ob ein zulässiger Interrupt einen bereits laufenden unterbrechen darf. Die Unterpriorität entscheidet nur, wer gewinnt, wenn mehrere Anforderungen gleichzeitig pending und zulässig sind. Im Lehrmodell sind vier Prioritätsbits als 2+2 aufgeteilt. Das ist eine gewählte Gruppierung, nicht automatisch die HAL-Voreinstellung. `__NVIC_PRIO_BITS` nennt die Anzahl implementierter Bits. CMSIS und HAL erwarten API-Werte. Die rohe Registerdarstellung hat eigene Bitpositionen.
A = (1,0) und B = (1,3) haben dieselbe Präemptionspriorität 1. Sind beide pending, gewinnt A wegen der kleineren Unterpriorität. Läuft B bereits, kann A es deshalb nicht verdrängen. Die Tabelle auf der Folie stellt genau diese beiden Fälle gegenüber.
NVIC-Prioritäten regeln, welcher zulässige Interrupt laufen darf. Warten Tasks auf eine geteilte Ressource, entsteht ein anderes Prioritätsproblem, das wir hier kurz vorbereiten.

Folie 44 / 59

Prioritätsinversion (Teaser)

- **Task-Kontext** (nicht ISR): Low hält Ressource, High wartet, Medium verzögert Low
- Pro Kern maximal eine Spur gleichzeitig *Running*
- Mutex-Erwerb/-Freigabe markieren; **kein** blockierender Mutex in einer ISR
- Vollständige Behandlung (Priority Inheritance): VL4



- Teaser: Mars Pathfinder (1997) — Details in VL4

Notizen 44 / 59

Ein Mutex ist eine Sperre mit Eigentümerschaft für eine gemeinsam genutzte Ressource. Low besitzt sie. High fordert sie an und blockiert. Medium kann Low verdrängen und damit indirekt High verzögern. Die Zeitachse zeigt das chronologisch: Low läuft und hält die Ressource, High wird blockiert, Medium läuft, danach kann Low freigeben, und High läuft. Auf einem Kern ist immer nur eine Running-Spur aktiv. Dieser Effekt ist kein Fall von NVIC-Subpriority. Eine ISR wartet nicht auf einen blockierenden Task-Mutex. Priority Inheritance hebt vorübergehend die Priorität des Eigentümers an und wird später behandelt. Mars Pathfinder 1997 bleibt ein historisches Teaserbeispiel. Eine Detailchronologie wird hier nicht erfunden. Beim eigentlichen Interruptwechsel optimiert Cortex-M den Registertransfer durch Tail-Chaining.

Folie 45 / 59

Tail-Chaining: ARMs Geheimwaffe

- Zwei Interrupts A und B treten kurz nacheinander auf
- Ohne Optimierung: Entry A → A → Return A → Entry B → B → Return B
- Mit Tail-Chaining: Entry A → A → Übergang zu anstehendem zulässigem B → B → Return
- Idealisiertes Cortex-M4-Modell: Return+neuer Entry $\approx 12 + 12$ vs. Übergang ≈ 6 Zyklen
  → **18 Übergangszyklen gespart** (nicht „jede Latenz = 6“)

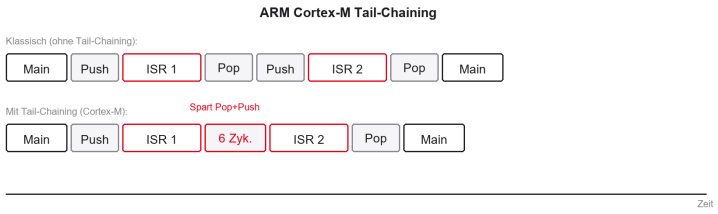


Figure 1: Tail-Chaining: Übergang mit und ohne Tail-Chaining, nur die idealisierten M4-Übergangszahlen

- Bedingung: B steht beim Exception Return an **und** darf angenommen werden

Notizen 45 / 59

Entry ist der Ausnahme-Eintritt, Return der Rücksprung. Stacking sichert den Frame, Unstacking stellt ihn wieder her. Ohne Tail-Chaining würde der Kern zwischen zwei Handlern den Zustand zur Thread-Rückkehr herstellen und ihn für den nächsten Handler sofort erneut sichern. Beim Tail-Chaining bleibt der passende Kontext liegen, und der Kern geht zum anstehenden zulässigen Handler über.
Im idealisierten Cortex-M4-Modell kosten Eintritt und erneuter Eintritt 12+12 = 24 Zyklen, der optimierte Übergang 6. Die Ersparnis sind 18 Übergangszyklen. Handlerlaufzeiten und die Pinlatenz sind darin nicht enthalten. B muss beim Rückkehrversuch anstehen und zulässig sein. Verschachtelte Präemption und Late Arrival, ein spät eintreffender dringenderer Interrupt während des Eintritts, sind andere Fälle. Die beiden Kerne verketten ihre Handler nicht miteinander. Die Abbildung vergleicht den Übergang mit und ohne Tail-Chaining.
Die gezeigten Zykluszahlen beschreiben einen Kernmechanismus unter Annahmen. Unser Board ergänzt zwei Kerne, Speicher und Busse, deren konkrete Eigenschaften wir jetzt einordnen.

Folie 46 / 59

Exkurs Hardware: Der STM32H755 / NUCLEO-H755ZI-Q

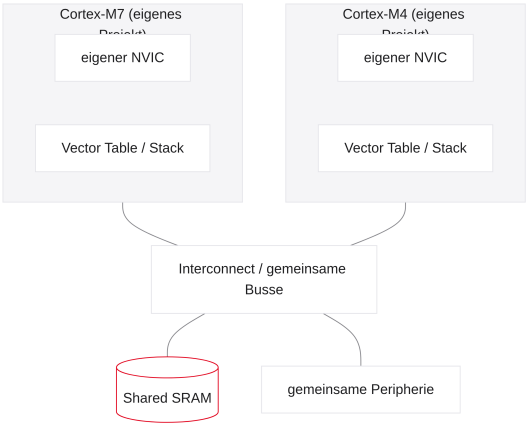
- **Board:** NUCLEO-H755ZI-Q (Nucleo-144, MB1363, internes SMPS $-\mathrm{Q}$)
- **MCU auf dem Board:** STM32H755ZIT6 (LQFP144, 2 MB Flash) — nicht mit dem Boardnamen verwechseln
- Cortex-M7 (Spec bis 480 MHz): Leistung, Caches, FPU
- Cortex-M4 (Spec bis 240 MHz): oft für Regelung genutzt
- Beide teilen Chip, Busse und teilweise Speicher → **Interferenz möglich**

Notizen 46 / 59

Der Boardname NUCLEO-H755ZI-Q, die Boardreferenz MB1363 und der MCU-Typ STM32H755ZIT6 sind drei verschiedene Bezeichnungen. LQFP144, Low-profile Quad Flat Package mit 144 Anschlüssen, beschreibt das Gehäuse. Flash ist der nichtflüchtige Programmspeicher, hier mit 2 MB angegeben. SMPS, Switched-Mode Power Supply, ist die getaktete Stromversorgung der Q-Variante. Cache ist ein schneller Zwischenspeicher, die FPU die Gleitkommaeinheit. Die spezifizierten Maxima 480 MHz und 240 MHz sind keine Aussage über die heute gewählte Konfiguration. Die Lehrannahme nennt 400 MHz und 200 MHz. PE1 ist Port E, Pin 1. LD2 ist die gelbe LED. Bei der beschriebenen Bestückung schaltet HIGH sie ein. Andere LEDs hängen von Boardrevision und Brücken ab: LD1 kann über Jumper an PB0 oder PA5 liegen, LD3 ist rot an PB14. Eine `.ioc` ist die Konfigurationsdatei von STM32CubeMX. Im Repository liegt keine.
Zwei Kerne erlauben eine Aufgabenverteilung, teilen aber trotzdem Ressourcen.

Folie 47 / 59

Warum Dual-Core (AMP) in der Echtzeit?



Lehrbeispiel-Verteilung:

- M7: Ethernet, GUI, Filter
- Latenz kann durch Caches schwanken

Lehrbeispiel-Verteilung:

- M4: Motor-PWM, Timing-Loops
- oft besser vorhersagbar (kein D-Cache)
- **Keine Garantie** „M4 = hard, M7 = soft“ — beide können Echtzeitaufgaben tragen
- Konflikt um Shared SRAM/Peripherie → Zuständigkeit + Synchronisation (HSEM)
- HSEM: Zugriff/Benachrichtigung — **keine** Cache-Kohärenz

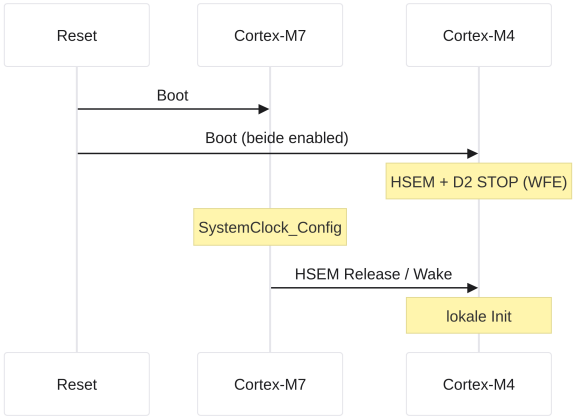
Notizen 47 / 59

AMP, Asymmetric Multiprocessing, bedeutet, dass auf den Kernen unterschiedliche Software laufen kann. Eine mögliche Lehrverteilung legt Ethernet, eine grafische Oberfläche GUI und Filter auf den M7 und PWM-Regelung auf den M4. Aus dem Kernnamen folgt keine Echtzeitklasse. Beide Kerne teilen Busse und Teile des Speichers, deshalb kann die Last des einen die Zeit des anderen beeinflussen. Peripherie braucht eine klare Zuständigkeit. Eine vereinbarte Datenübergabe macht die Zusammenarbeit prüfbar. Ein fehlender Datencache auf dem M4 beseitigt weder Flash-Waitstates noch Buskonflikte oder lange ISRs. HSEM kann Besitz und Benachrichtigung koordinieren und stellt keine Cache-Kohärenz des M7 her. Bevor beide Programme arbeiten können, muss ihr Start abgestimmt sein.

Folie 48 / 59

Gewählte Bootfolge (CubeIDE-Typ)

- Beschreibt die **häufige CubeIDE-/Template-Sequenz** bei beiden aktivierten Kernen — keine universelle Reset-Regel
- Übliches Template: M4 schaltet die HSEM-Benachrichtigung scharf und geht in **D2-Stop** (WFE). M7 wartet auf diesen Zustand, setzt die gemeinsamen Takte und gibt das HSEM frei. Dadurch wird M4 geweckt.
- Das ist **Software-Sync nach gemeinsamem Boot**, kein „M4 ab Reset festgehalten“
- Echtes Hold-from-Reset nur über Option Bytes (z. B. BCM4/BCM7) — nicht beiläufig ändern



Notizen 48 / 59

Die Abbildung hat die Spuren Reset, Cortex-M7 und Cortex-M4. Unter der angenommenen Bootkonfiguration starten beide Kerne. Ein Image ist der eigene Firmwareinhalt eines Kerns. Der M4 schaltet die HSEM-Benachrichtigung scharf und geht in D2-Stop. D2 ist eine Spannungs- und Takt-Domäne des Chips. WFE, Wait For Event, wartet dort auf ein Ereignis. Der M7 wartet auf den vorgesehenen D2-Zustand, konfiguriert die gemeinsamen Takte und gibt das HSEM frei. Diese Freigabe benachrichtigt den M4. Der M4 löscht gegebenenfalls das Flag und setzt seine lokale Initialisierung fort. Die Benachrichtigung läuft also vom M7 zum wartenden M4, nicht als unbestimmtes „M4 meldet sich“.
HSEM-FastTake und ein späteres Take/Release mit expliziter ProcessID sind verschiedene Protokolle. Das übernommene Template kann andere Besitzerkennungen nutzen als die Lehr-ProcessIDs `0x37` und `0x34`. Ein Software-Wartezustand ist etwas anderes als ein per Option-Bytes deaktivierter Kernstart. BCM4 und BCM7 gehören zu dieser Option-Byte-Bootkonfiguration und werden hier nicht verändert.
Die benötigten Projektteile und Einstellungen werden mit der Toolchain erzeugt.

Folie 49 / 59

Einführung in die Tools: STM32CubeIDE

- Empfohlene Lehrumgebung ab CubeIDE 2.0: STM32CubeMX konfiguriert und erzeugt den Code, STM32CubeIDE importiert, übersetzt und debuggt
- In CubeIDE 1.x steckte die Konfiguration noch in der IDE
- Pin-Zuweisung, Clock-Tree und HAL-Grundgerüst kommen aus dem Generator
- Der Generator beweist keine Zeitgrenze

Notizen 49 / 59

IDE steht für Integrated Development Environment, integrierte Entwicklungsumgebung. Die Kette ist Konfiguration, Codegenerierung, Übersetzung, Linken, Flashen und Debuggen. STM32CubeMX beschreibt Pins, Peripherie und den Clock-Tree. STM32CubeIDE bearbeitet das Programm und bereitet die Ausführung vor. Empfohlene Lehrumgebung, nicht als bereits erprobter Laborstand behauptet: ab CubeIDE 2.0 konfiguriert und erzeugt CubeMX getrennt, CubeIDE importiert, baut und debuggt. In CubeIDE 1.x war das Konfigurationswerkzeug integriert. Welche Installation im Raum steht, ist im Repository nicht festgelegt.
Der Generator ersetzt nicht die Lektüre der Hardwarebeschreibung und erzeugt keinen Timing-Nachweis. Eine `.ioc` enthält die Konfiguration, nicht ein gemessenes Laufzeitverhalten. ST-LINK ist die Debug-Schnittstelle auf dem Board.
Der erzeugte Code verwendet überwiegend die HAL-Schnittstelle.

Folie 50 / 59

Das HAL-Konzept

- Nicht direkt in Register tippen (Standardfall)
- HAL: lesbar und oft portabel innerhalb der STM32-Familie
- Nachteil: leichter Aufruf-Overhead
- Extreme Echtzeit: LL oder direkte Register
- Vergleich nur bei **gleichwertiger** Initialisierung/Statusbehandlung

```
/* HAL (nach vollständiger Init) */
HAL_TIM_Base_Start_IT(&htim2);

/* LL: Enable allein ≠ komplette Init */
LL_TIM_EnableIT_UPDATE(TIM2);
LL_TIM_EnableCounter(TIM2);
```

Notizen 50 / 59

HAL, Hardware Abstraction Layer, kapselt Initialisierung, Statusprüfung und wiederkehrende Abläufe in Funktionen. Auch HAL spricht am Ende Register an. LL, Low Layer, ist die registernahe Treiberschnittstelle und bedeutet hier nicht Liu-Layland. Der HAL-Aufruf `HAL_TIM_Base_Start_IT` setzt eine fertige Initialisierung voraus. Zwei LL-Enable-Schritte allein sind keine gleichwertige Gesamtinitialisierung und keine automatische NVIC-Freigabe.
Ein fairer Laufzeitvergleich braucht dieselbe Funktion, dieselbe Timerkonfiguration, dieselbe Flagbehandlung, dieselben Compileroptionen und dieselben Messendpunkte. Eine Overheadzahl wird hier nicht erfunden. Direkte Register sind nicht allein dadurch echtzeitkorrekter.
Ein fairer Vergleich setzt voraus, dass HAL- und LL-Variante dieselbe Timerkonfiguration, dieselbe Flaglöschung und denselben Messpunkt verwenden. Sonst misst man unterschiedliche Aufgaben.
Eigene Anwendung und generierter Initialisierungscode müssen im Projekt sauber zusammenpassen.

Folie 51 / 59

Projektstruktur und Code-Generierung

- Pro Core eigener Ordner (CM4 / CM7)
- Je eigene `main.c` und `stm32h7xx_it.c`
- Eigener Code nur zwischen `USER CODE BEGIN` und `USER CODE END`
- Beim erneuten Generieren bleiben markierte USER-Abschnitte erhalten; andere Dateien und Abschnitte können andere Regeln haben
- Keine zweite `while(1)` in USER-Blöcken schachteln; Callbacks nicht doppelt definieren

Notizen 51 / 59

CM4 und CM7 sind die Projektteile für Cortex-M4 und Cortex-M7, nicht die Kerne selbst. Jeder Teil hat eine `main.c` und die Handlerdatei `stm32h7xx_it.c`. Die Anwendungsschleife steht in `main`, die Interruptweiterleitung in der Handlerdatei. `USER CODE BEGIN` und `USER CODE END` sind Generator-Markierungen. Code dazwischen bleibt beim erneuten Generieren erhalten, wenn der passende Erhaltungsmodus gilt. Kritisch ist die Codegenerierung, nicht schon jedes Speichern in der Oberfläche. Dateien und Abschnitte können unterschiedliche Erhaltungsregeln haben.
Der Schleifeninhalt gehört in den vorhandenen Schleifenblock, ein Callback einmal in seinen USER-Block. Eine zusätzliche verschachtelte Endlosschleife und eine zweite Definition derselben Funktion würden den Ablauf zerstören.
Das erste Labor prüft diesen Grundaufbau durch eine LED-Reaktion.

Folie 52 / 59

Laboraufgabe 1: Hello World (LED Blink)

- Ziel: Toolchain prüfen, Cortex-M4 nutzen
- Schritte:
  1. In CubeMX das Board NUCLEO-H755ZI-Q anlegen und erzeugen; in CubeIDE 1.x geht das in der IDE
  2. LED **PE1** (LD2 gelb auf MB1363) als `GPIO_Output` für CM4
  3. Code generieren, in CubeIDE importieren; GPIO- und Clock-Init prüfen
  4. In `main.c` (CM4) toggeln

```
/* USER CODE BEGIN 3 - Inhalt der vorhandenen while(1) */
HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
HAL_Delay(500); /* blockiert; kompletter LED-Zyklus ≈ 1 s */
/* USER CODE END 3 */
```

Notizen 52 / 59

Voraussetzungen sind das richtige Board, Projektteile für beide Kerne nach der gewählten Bootfolge, die Systeminitialisierung auf dem M7 und eine CM4-Anwendung zunächst ohne FreeRTOS. `GPIO_Output` auf PE1 macht den Pin zum Ausgang. `GPIOE` ist der Port-E-Baustein, `GPIO_PIN_1` die Pinmaske für Pin 1. `HAL_GPIO_TogglePin` schaltet den Pegel um. Der Code gehört in die vorhandene CM4-Schleife. `HAL_OK` wäre der Erfolgscode eines HAL-Aufrufs, falls Sie den Rückgabewert prüfen. `HAL_Delay(500)` wartet blockierend auf die HAL-Zeitbasis. Interrupts können dabei weiterlaufen. Zwei Umschaltungen bilden einen vollständigen Ein- und Aus-Zyklus, ungefähr 1 s beziehungsweise 1 Hz, zuzüglich Code- und Tickaufwand. Eine exakte Dauer von 500 ms garantiert `HAL_Delay` nicht. Gebaut und geladen werden die erforderlichen Images. Die Bootfreigabe folgt der beschriebenen HSEM-Sequenz. Weitere Debugfunktionen werden nur behauptet, wenn sie im Labor tatsächlich gezeigt werden. Danach fügen wir Hintergrundarbeit ein und beobachten die Zeitverschiebung.

Folie 53 / 59

Laboraufgabe 2: Das Polling-Desaster

- Ziel: Grenzen der Super-Loop spüren
- `Heavy_Load()` so schreiben, dass der Compiler die Arbeit nicht entfernt (Ergebnis nutzen)
- **Konstante** Last verlängert den Abstand konstant (noch kein Jitter); für Jitter bewusst **variable** Lasten

```
/* in vorhandener while(1), USER CODE BEGIN 3 */
Heavy_Load(800000); /* Iterationen ≠ bekannte ms-Dauer */
HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
HAL_Delay(500);
```

- Beobachtung: Warum weicht der sichtbare Takt vom Soll ab?

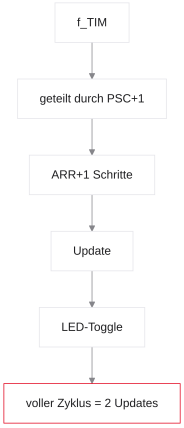
Notizen 53 / 59

`Heavy_Load` ist absichtlich erzeugte Rechenlast. Die Zahl 800000 ist ein Iterationsparameter, keine bekannte Millisekundenangabe. Ein benutztes Ergebnis hindert den Compiler daran, die unbenutzte Arbeit zu entfernen. `volatile` allein ist kein allgemeiner Optimierungsstopp. Wenn Sie Beispielcode ergänzen, verwenden Sie korrekt begrenzte vorzeichenlose Arithmetik und eine nachvollziehbar genutzte Ausgabe. Compiler, Optimierungsgrad und Kernkonfiguration gehören ins Protokoll.
Der Abstand zwischen Umschaltungen ist näherungsweise 500 ms plus Lastdauer plus sonstiger Aufwand. Konstante Last verlängert die Periode vor allem konstant. Variable Last kann Schwankung erzeugen. Ohne Messung gibt es keinen Messwert.
Ein Beispiel mit vorzeichenloser Zählschleife bleibt ein Muster, solange Compiler und Optimierung nicht protokolliert sind. Die beobachtete LED-Periode ist dann die Summe aus Delay, Last und restlichem Schleifenaufwand.
Die zeitliche Auslösung wird nun vom Main-Kontrollfluss auf einen Hardwaretimer verlagert.

Folie 54 / 59

Laboraufgabe 3: Rettung durch den Timer

- Ziel: periodische LED-Flanken per TIM2-IRQ
- In der `.ioc` (CM4): TIM2, Clock Source Internal; NVIC „TIM2 global interrupt“
- Vor der Schleife: `HAL_TIM_Base_Start_IT(&htim2)` mit Fehlerprüfung



- Upcounter-Modell:

$$f_{\text{UPD}} = \frac{f_{\text{TIM}}}{(PSC+1)\,(ARR+1)}$$

- Lehrbeispiel (nur wenn $f_{\text{TIM}} = 100\,\text{MHz}$ in der `.ioc` gilt): $PSC = 9999$, $ARR = 4999 \rightarrow 2$ Hz Update (500 ms), LED-Zyklus ≈ 1 Hz
- f_{TIM} aus Clock-Tree (APB-Prescaler/`TIMPRE`: oft $2 \times$ PCLK bei Prescaler $\neq 1$) — nicht automatisch `SystemCoreClock`
- TIM2 ohne Repetition Counter: $(\text{RCR} + 1) = 1$

Notizen 54 / 59

TIM2 ist die zweite Timerinstanz des Mikrocontrollers. Ein Timer zählt Hardwaretakte, auch wenn die Hauptschleife andere Arbeit ausführt. PSC, Prescaler, teilt den Timer-Kern-Takt f_TIM durch PSC+1. Der Zähler läuft im Upcounter von null bis ARR, dem Auto-Reload Register. Weil null mitzählt, umfasst eine Runde ARR+1 Schritte. Beim Überlauf entsteht ein Update. Damit gilt f_UPD = f_TIM / ((PSC+1)*(ARR+1)). TIM2 hat keinen Repetition Counter. Die allgemeine Formel mit RCR darf hier nicht zusätzlich angewendet werden. PCLK ist der Peripheriebustakt. TIMPRE wählt, wie der Timer-Takt aus dem Bus abgeleitet wird. SystemCoreClock ist nur die Softwarevariable für den Kerntakt.
Unter der ausdrücklich angenommenen Timerfrequenz von 100 MHz und mit PSC = 9999 beträgt der Zähltakt 100 MHz / 10000 = 10 kHz. ARR = 4999 ergibt 5000 Schritte. Es entstehen 10000/5000 = 2 Updates je Sekunde, also alle 500 ms. Zwei Umschaltungen bilden einen LED-Zyklus von etwa 1 Hz. Das Flussdiagramm trennt diesen Updateabstand vom vollständigen LED-Zyklus.
Ein APB-Takt von 100 MHz kann bei Teiler 2 und passender Timerwahl 200 MHz Timer-Kern-Takt ergeben. Dieselben Register ergäben dann 4 Hz Update, 250 ms Abstand und 2 Hz LED-Zyklus. Für wieder 2 Hz Update wäre zum Beispiel PSC = 19999 bei ARR = 4999 passend. Beide Rechnungen sind bedingt. Ohne `.ioc` und Messung ist f_TIM nicht nachgewiesen.

Notizen, Fortsetzung 54 / 59

Das Update muss als Interrupt zulässig sein. Der NVIC muss TIM2 freigeben. `TIM2_IRQHandler` ruft den HAL-Handler auf, dieser den Callback. Wir starten den initialisierten Timer vor der Schleife und prüfen das Ergebnis. `HAL_OK` bedeutet Erfolg.
Als Nächstes legen wir im Callback fest, dass nur das TIM2-Update die LED umschaltet.

Folie 55 / 59

Laboraufgabe 3: Der Callback-Code

- Toggle/`HAL_Delay` aus der Main-Schleife entfernen — dort z. B. nur noch `Heavy_Load()`
- Callback **einmal** im vorgesehenen USER-Block (nicht doppelt definieren):

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1); /* LD2 gelb */
    }
}
/* USER CODE END 4 */
```

- Messbare Aussage: unter dieser Konfiguration periodische Flanken trotz Hintergrundlast — **kein** automatischer harter Fristnachweis

Notizen 55 / 59

Die LED wird nur noch im Callback umgeschaltet. Die Umschaltung und der kurze `HAL_Delay` müssen aus der Hauptschleife entfernt werden, sonst überlagern sich zwei Taktquellen. Die Funktion `HAL_TIM_PeriodElapsedCallback` erhält einen Zeiger auf ein `TIM_HandleTypeDef`. `htim->Instance` vergleicht die konkrete Timerinstanz mit TIM2, weil derselbe Callback mehrere Timer bedienen kann. `HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1)` schaltet LD2 um.
Die Funktion läuft im IRQ-Kontext auf dem CM4 und wird über den generierten Handler und HAL aufgerufen. `HAL_Delay` und `Heavy_Load` gehören nicht in den Callback. Lange Hauptarbeit kann unterbrochen werden, wenn IRQs zulässig sind. Maskierung, höhere Handler und Buslast bleiben relevant. Bei zu langsamer Bedienung können Updateereignisse zusammenfallen. Ein einzelnes Flag ist kein unbegrenzter Ereignisspeicher.
Die LED-Auslösung hängt nun nicht mehr vom Ende der Hauptschleife ab. Wie groß die verbleibenden Verzögerungen und Schwankungen sind, lässt sich mit klar definierten Messpunkten untersuchen.

Folie 56 / 59

Laboraufgabe 4 (Bonus): Drei Oszi-Messungen

Messung	Was	Aussage
A	Puls um `Heavy_Load()`	verstrichene Dauer inkl. möglicher IRQs
B	periodische Flanken Main vs. TIM2-Callback	gleiche Sollperiode; konstante vs. variable Last getrennt
C	Stimulus/Referenz vs. früher Handler-Marker	Anfang/Ende nennen; HAL-Callback ≠ reine Entry-Zeit

- Pin z. B. PA0 erst nach Pinbelegung prüfen; Trigger, Zeitbasis, Sampleanzahl notieren
- Schema: Min/Max/Mittel/Peak-to-Peak — Zahlen **erst nach Messung** eintragen
- Eine optisch ruhige LED ist kein Fristnachweis. Zahlen erst nach der Messung eintragen



Lehrskizze ohne Messkurve. Die Tabelle bleibt leer.

Notizen 56 / 59

Oszi ist das Oszilloskop. Alle Kanäle brauchen denselben Masse- und Zeitbezug. Sie stellen Messkanal, Flanke, Trigger, Zeitbasis und Abtastung ein und notieren sie. PA0 ist nur ein Beispiel, nachdem die konkrete Pin- und Boardbelegung geprüft wurde. Der Taster kann je nach Bestückung woanders liegen. Messung A setzt Marker vor und nach `Heavy_Load`. Sie erfasst die verstrichene Dauer einschließlich möglicher IRQ-Unterbrechungen und des Markeraufwands, nicht automatisch reine CPU-Zeit. Messung B vergleicht Hauptschleifen- und Timer-Umschaltungen unter derselben Zielperiode. Konstante und wechselnde Last werden getrennt betrachtet. Perioden werden einzeln aus Zeitstempeln berechnet. Messung C benutzt dieselbe auslösende Referenz und einen frühen Handler-Marker. Ein Marker erst im Callback enthält die zusätzlichen Softwareanteile. Ein freilaufendes unabhängiges Signal ist kein gültiger gemeinsamer Startpunkt.
Die Tabelle für Minimum, Maximum, Mittelwert und Peak-to-Peak bleibt leer, bis N Läufe unter dokumentierter Konfiguration vorliegen. Debughalt, Auflösung und Markeraufwand verändern das Ergebnis. Es gibt in diesen Unterlagen keine Boardmesswerte.
Mit diesen Beobachtungen werden die Lernziele noch einmal geprüft.

Folie 57 / 59

Zusammenfassung des Tages

- Echtzeit = Rechtzeitigkeit (Fristvertrag), nicht pure Rechenleistung
- Hard/Firm/Soft: Konsequenz und Nutzen nach Miss spezifizieren
- Messmaximum $\neq$ sichere WCET-Schranke; Jitter ≠ Varianz (Metrik nennen)
- Polling kann Deadlines erfüllen — lange unzerlegte Arbeit oft nicht
- Interrupts/RTOS helfen strukturieren; Deadlines weiterhin nachweisen
- Exception Entry ≠ Taskwechsel; NVIC/Tail-Chaining mit Annahmen
- Labor: NUCLEO-H755ZI-Q, TIM2-Pfad messen statt „Jitter ≈ 0“ behaupten

Notizen 57 / 59

Der Mittelwert 6 ms bei System A verhindert den Miss von 18 ms bei D = 10 ms nicht. Hart, fest und weich unterscheiden sich darin, was nach der Frist mit Anforderung und Nutzen geschieht. Am Zeitstrahl sind 3 ms CPU-Bedarf und 7 ms Antwortzeit verschiedene Größen. Eine konstante Periode von 550 ms statt 500 ms ist eine systematische Abweichung, die Spanne 0,4 ms im Lehrbeispiel ist Periodenschwankung. Eine kurze ISR garantiert nicht die spätere Verarbeitung in der Hauptschleife. Exception Entry sichert den Ausnahme-Eintritt. Ein Taskwechsel wählt zusätzlich einen anderen RTOS-Task und passiert im heutigen Labor nicht. Zwei Timer-Updates ergeben erst einen vollständigen LED-Zyklus.
Frage: Beweisen vier erfolgreiche Antworten eine harte Frist? Nein, weil ein späterer Miss den allgemeinen Anspruch widerlegt. Frage: Laufen Main und ISR auf einem Kern parallel? Nein. Frage: Ist 12 Zyklen die Pin-bis-Callback-Zeit? Nein, das ist das idealisierte M4-Eintrittsmodell. Frage: Ist APB gleich Timer-Kern-Takt? Nicht automatisch. Der zweite Kern und ein optisch ruhiges Blinksignal liefern allein noch keinen harten Nachweis.
Ein einzelner Timer zeigt das Grundprinzip. Für mehrere Aufgaben müssen wir Ausführungsbedarf, Fristen und CPU-Zuteilung systematisch zusammendenken.

Folie 58 / 59

Ausblick auf Termin 2

- Bare-Metal mit vielen IRQs skaliert nicht ewig (Spaghetti-Flags)
- Nächste Woche: Scheduling-Theorie (Task-Modell, Utilization, RMS/EDF)
- Praxis: eigener einfacher Round-Robin-Scheduler auf dem STM32
- Vorbereitung: heutiges Projekt mitbringen

Notizen 58 / 59

Scheduling ist die Zuteilung von CPU-Zeit. Utilization ist die Auslastung, der Anteil der Zeit, den die Aufgaben beanspruchen. RMS, Rate-Monotonic Scheduling, vergibt feste Prioritäten nach der Periode: kürzere Periode, höhere Priorität. EDF, Earliest Deadline First, wählt den bereiten Job mit der nächsten absoluten Deadline. Round-Robin wählt reihum. Formeln folgen im nächsten Termin. Der eigene einfache

Notizen, Fortsetzung 58 / 59

Scheduler dort ist ein Lernschritt vor dem späteren FreeRTOS.
Bitte bewahren Sie die Konfigurationsdatei, die CM4- und CM7-Quellen, die Build- und Debugeinstellungen und ein Messprotokoll auf, sobald Sie es tatsächlich ausgefüllt haben. Leere Tabellen bleiben leer.
Die abschließende Quellenfolie zeigt, wie Hardware- und Modellbehauptungen geprüft werden.

Folie 59 / 59

Quellenhinweise (Hardware)

- ST, STM32H755-Datenblatt / RM0399 (NVIC ≈ 150 Kanäle/Kern, Busse, Timer)
- ST, UM2408 (NUCLEO-H7 MB1363, LED/Pinbelegung)
- ST, AN5361 / AN5557 (Dual-Core CubeIDE bzw. Architektur/Boot)
- ST, PM0214 (Cortex-M4 Exceptions / Stackframe)
- Arm CMSIS-Core NVIC; Arm Interrupt-Latency-Guide (Voraussetzungen idealer Zykluszahlen ≈ 12 Entry / 6 Tail-Chaining)
- Kopetz / Buttazzo: Begriffe und Scheduling (Ausgabe im Semesterapparat)

Notizen 59 / 59

ST ist der Hersteller STMicroelectronics. RM, Reference Manual, beschreibt Peripherie und System. UM, User Manual, beschreibt Board und Bedienung. AN, Application Note, beschreibt einen konkreten Arbeitsablauf. PM, Programming Manual, beschreibt den Kern. DS, Datasheet, nennt Bauteilgrenzen. UM2408 prüft LED und Taster. Das Datenblatt prüft die Maximalfrequenzen. RM0399 prüft Takte und Peripherie. PM0214 prüft den M4-Stackframe, PM0253 die M7-Details. Arm- und CMSIS-Dokumentation prüfen den NVIC. Die ST-Dokumentation zu CubeIDE und AN5361 prüft Toolchain und Boot. Eine konkrete Seiten- oder Abschnittsnummer wird hier nicht erfunden. Die im Kurs verwendete Revision prüfen Sie an dem Dokument, das Sie öffnen.
Bücher sind Theoriequellen. Die Zahlenbeispiele dieser Vorlesung sind Lehrmodelle. Laborwerte entstehen erst durch Messung. Mit präzisem Zeitvertrag, dokumentierter Konfiguration und nachvollziehbaren Messpunkten beginnt eine belastbare Beurteilung des Echtzeitverhaltens.